

Section 1

Problem Setup: Regression

Your Boss:

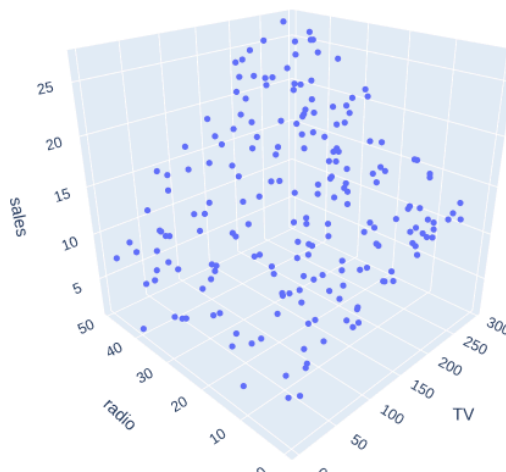
- "Should we increase TV advertising by 100\$?"
- "How confident are you?"

```
In [1]: import pandas as pd
import plotly.express as px

# Advertising isn't bundled with ISLP; this CSV is from statlearning.com
Advertising = pd.read_csv('Advertising.csv', index_col=0)

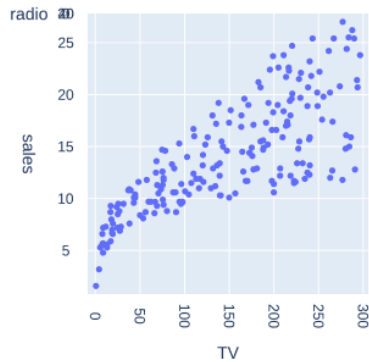
# Interactive 3-D plot of Sales vs TV and Radio: drag to rotate, scroll to zoom
fig = px.scatter_3d(Advertising, x='TV', y='radio', z='sales',
                    hover_data=['newspaper'], height=600)
```

```
In [2]: fig.update_traces(marker_size=3)
# View from the (0, 0) corner so both axes start at the front
fig.update_layout(scene_camera_eye=dict(x=-1.5, y=-1.5, z=1))
fig.show()
```



Now, let's view that plot from a direction perpendicular to the TV-Sales plane.

```
In [3]: # Show the plot from a direction perpendicular to the TV-Sales plane
# Look along the radio axis (from its low end), so TV runs left to right and
# Orthographic projection removes perspective, so the view is truly flat.
fig.update_layout(scene_camera=dict(eye=dict(x=0, y=-2.5, z=0),
                                         projection_type='orthographic'))
fig.show()
```



Linear Regression:

- Force/restrict the model to be linear: $y = mx + b$
- Which choice for m and b are "best"?
- How do we define/operationalize "best"?

Three points

For simplicity, we'll work with just three stores, spread out along the TV axis.
From here on, we only look at sales vs. TV.

```
In [4]: from regression_demos_helpers import pick_spread_points, fit_line, plot_line

# The same three stores/markets/points every run, at the 10th, 50th, and 90th
pts = pick_spread_points(Advertising, 'TV', n=3)
pts[['TV', 'sales']]
```

Out[4]:

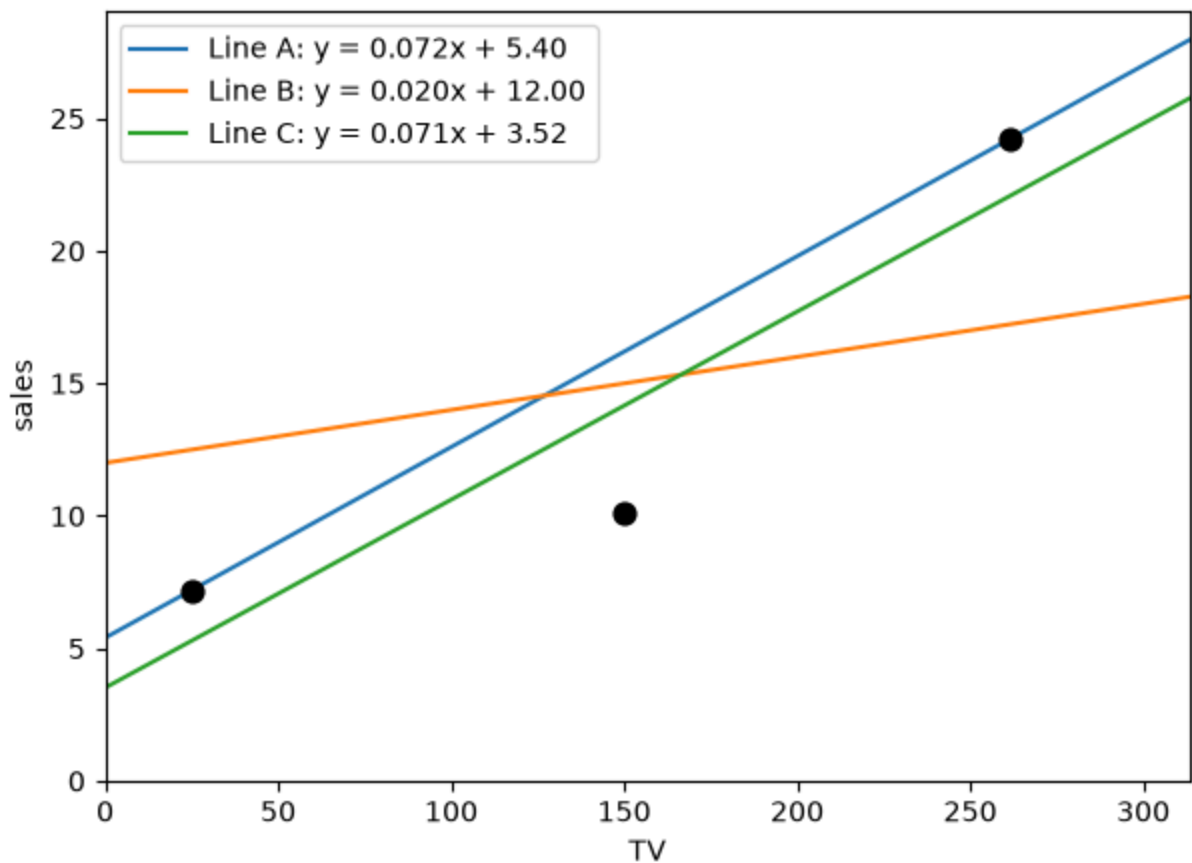
	TV	sales
107	25.0	7.2
158	149.8	10.1
62	261.3	24.2

Candidate lines

Each choice of slope m and intercept b gives a different line. Here are three candidates. Which looks best?

```
In [5]: m_best, b_best = fit_line(pts['TV'], pts['sales'])

# Each line is just a (slope, intercept) pair
lines = {
    'Line A': (0.072, 5.4),          # passes (almost) exactly through the two
    'Line B': (0.02, 12.0),         #
    'Line C': (m_best, b_best),     # the best-fit line; how we find it comes
}
plot_lines(pts, lines);
```



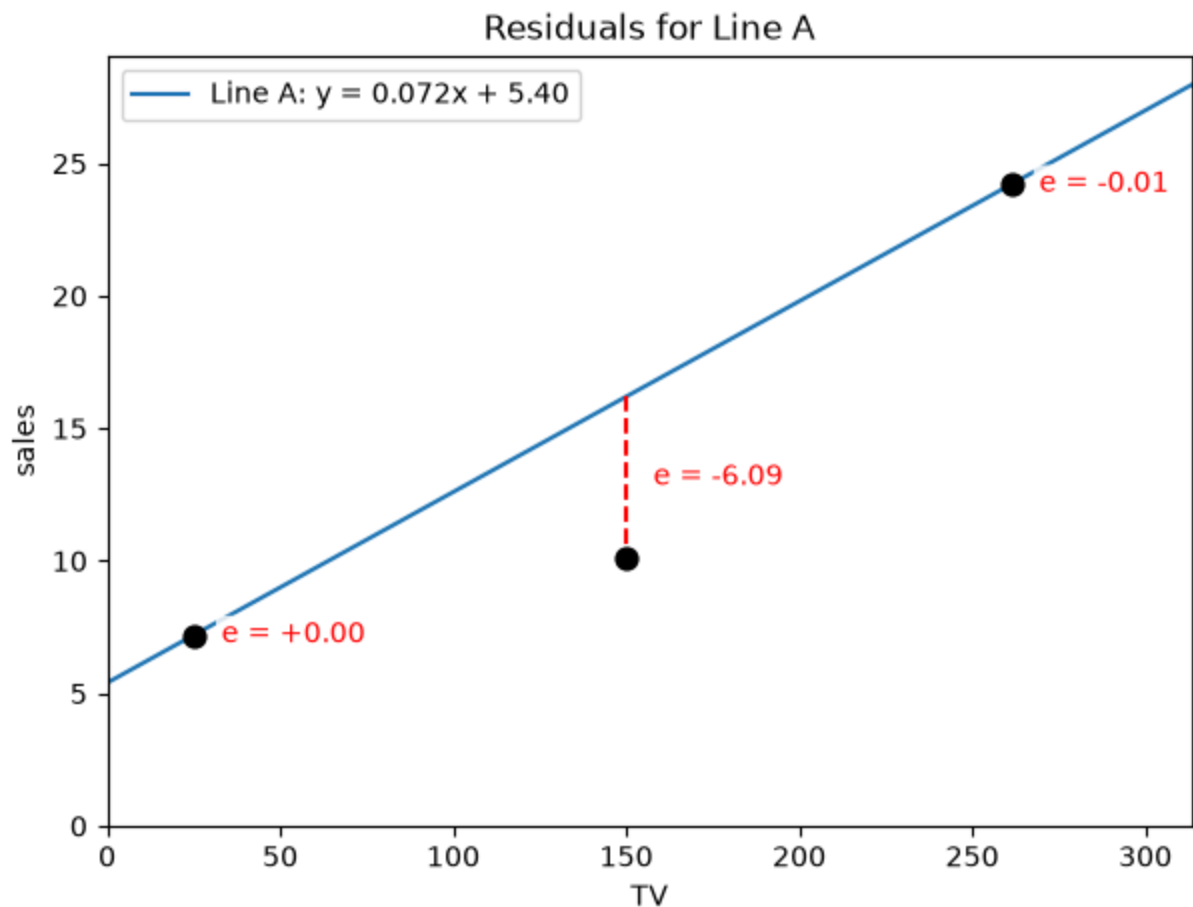
Residuals

For each point x_i , a line predicts $\hat{y}_i = mx_i + b$. The **residual** is the actual value minus the prediction:

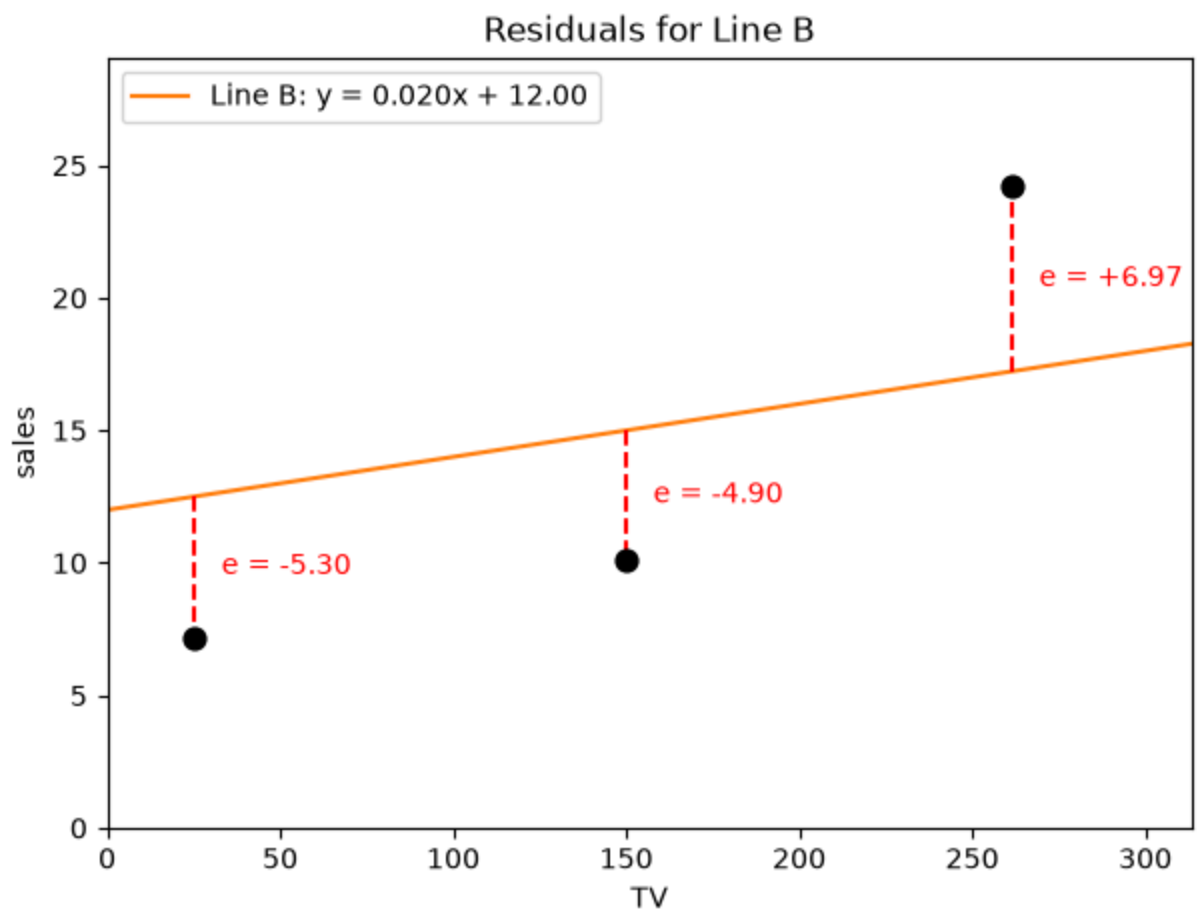
$$e_i = y_i - \hat{y}_i$$

A positive residual means the point is above the line; a negative one means it's below.

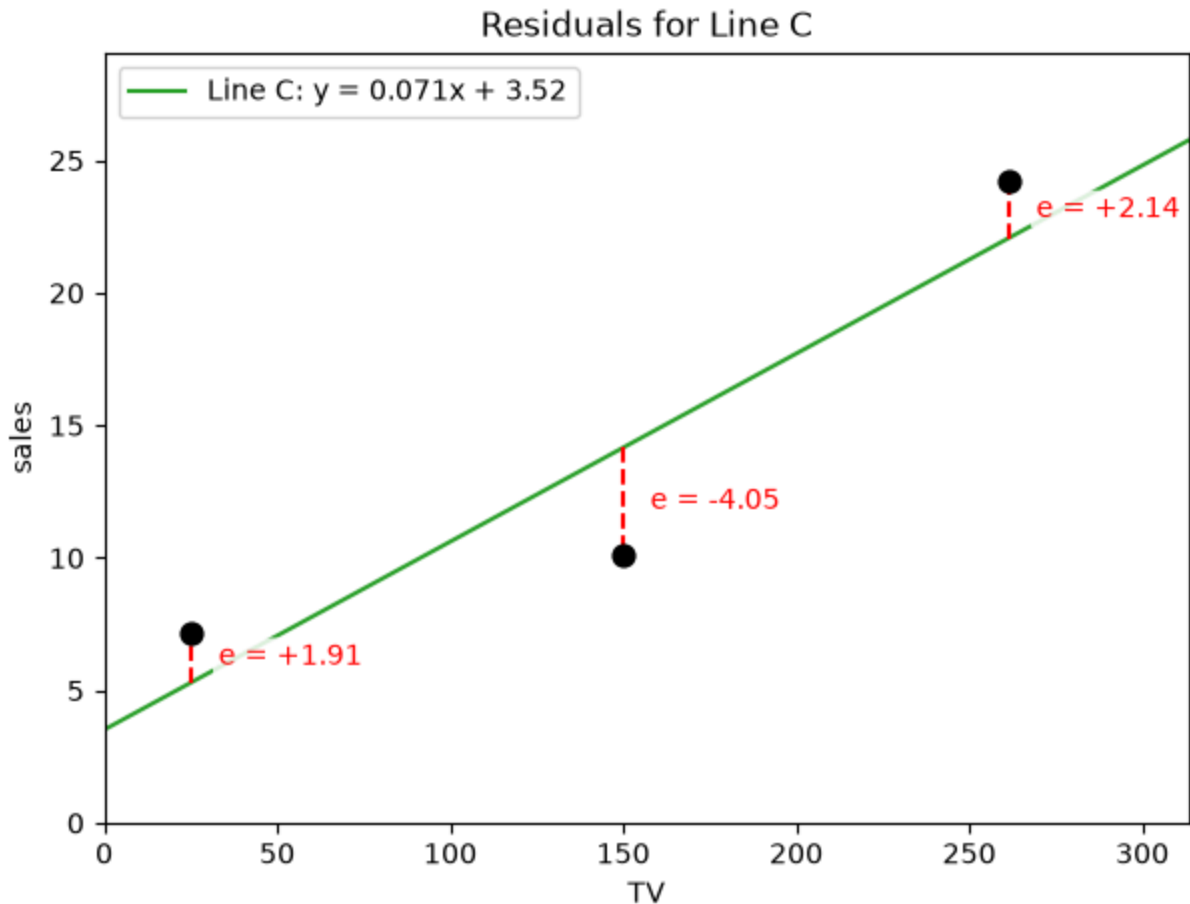
```
In [6]: plot_residuals(pts, lines, 'Line A');
```



```
In [7]: plot_residuals(pts, lines, 'Line B');
```



```
In [8]: plot_residuals(pts, lines, 'Line C');
```



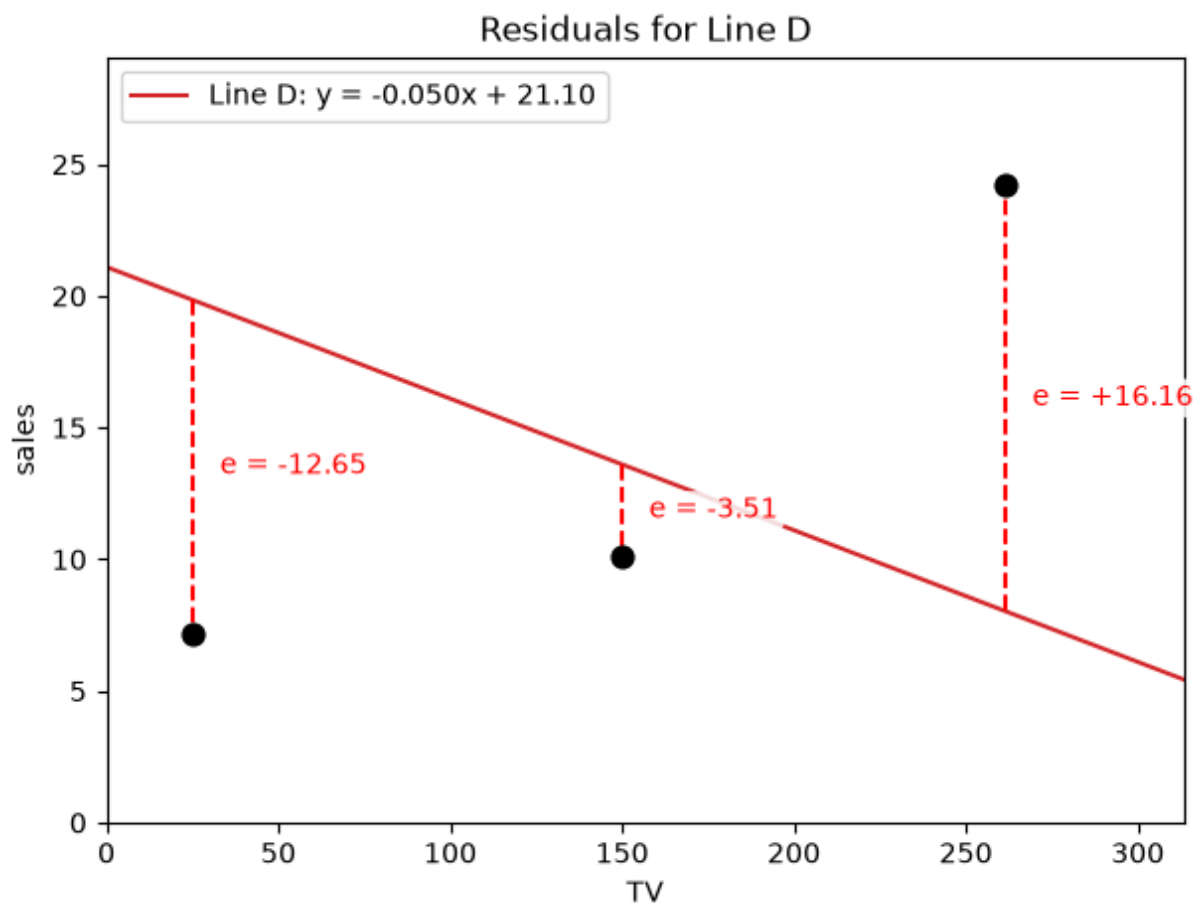
Discuss: Which line fits best? Line A goes right through two of the points; does that make it better than Line C? How could we combine the three residuals into one number that measures how well a line fits?

Proposal: Whichever line has the lowest *sum* of residuals is the best.

Look at the sum of residuals for this particular, new line, line D:

```
In [9]: from regression_demos_helpers import score_table, loss_explorer

# Line D: slope -0.05, positioned to pass through the mean point ( $\bar{x}$ ,  $\bar{y}$ )
m_D = -0.05
b_D = pts['sales'].mean() - m_D * pts['TV'].mean()
lines['Line D'] = (m_D, b_D)
plot_residuals(pts, lines, 'Line D');
```



```
In [10]: # Sum of residuals for each line
score_table(pts, lines)[['sum of e']]
```

```
Out[10]:
```

	sum of e
Line A	-6.10
Line B	-3.22
Line C	-0.00
Line D	0.00

Line D is obviously terrible, yet its residuals sum to 0, just like Line C's. (Any line through the mean point $(\bar{x}, \bar{y})$ does this.) So the sum of residuals can't be our measure.

Why did it fail? Positive and negative residuals cancel.

What we need: every residual should add to the "badness" score, whatever its sign. Any suggestions?

▼ Two natural options

$$\sum_i |e_i| \quad \text{RSS} = \sum_i e_i^2$$

RSS stands for *residual sum of squares* (An unfortunate name if you ask me. I'd prefer *sum of squared residuals*, but we're stuck with RSS).

```
In [11]: score_table(pts, lines)
```

```
Out[11]:
```

	sum of e	sum of e	sum of e ²
Line A	-6.10	6.10	37.03
Line B	-3.22	17.17	100.70
Line C	-0.00	8.10	24.61
Line D	0.00	32.33	433.65

The choice for how to represent "badness of fit" is often called the **Loss Function**, or often just "the loss*."

This is a choice. The two measures can disagree: sum of $|e|$ prefers Line A, while RSS prefers Line C. Squaring punishes big misses much more than small ones, so Line A's single miss of about 6 costs it about 37 in RSS, but only costs it $|-6.09| = 6.09$ in sum of $|e|$.

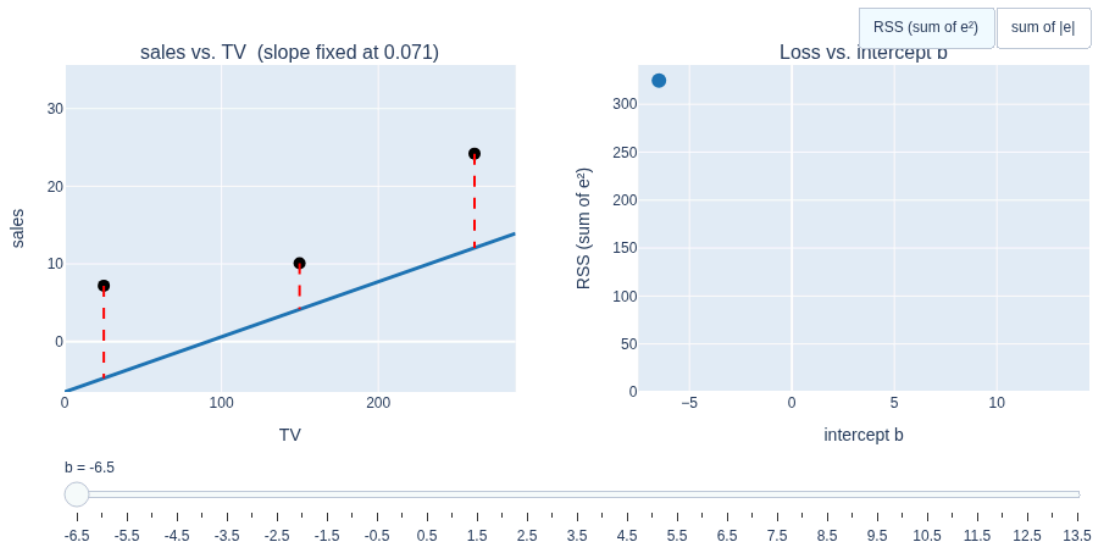
When we measure the loss (think "badness of fit") using RSS, we call that *Least squares* regression. This is the most common loss we'll use in the course for fitting lines to data.

Sweeping the intercept

Fix the slope at Line C's value and slide the intercept b . Each slider step shifts/translate the line on the left vertically and adds one point to the loss curve on the right. The buttons switch between RSS and sum of $|e|$.

```
In [41]: import numpy as np

b_values = np.linspace(b_best - 10, b_best + 10, 41) # 41 intercepts centered at b_best
loss_explorer(pts, 'b', b_values, m=m_best)
```

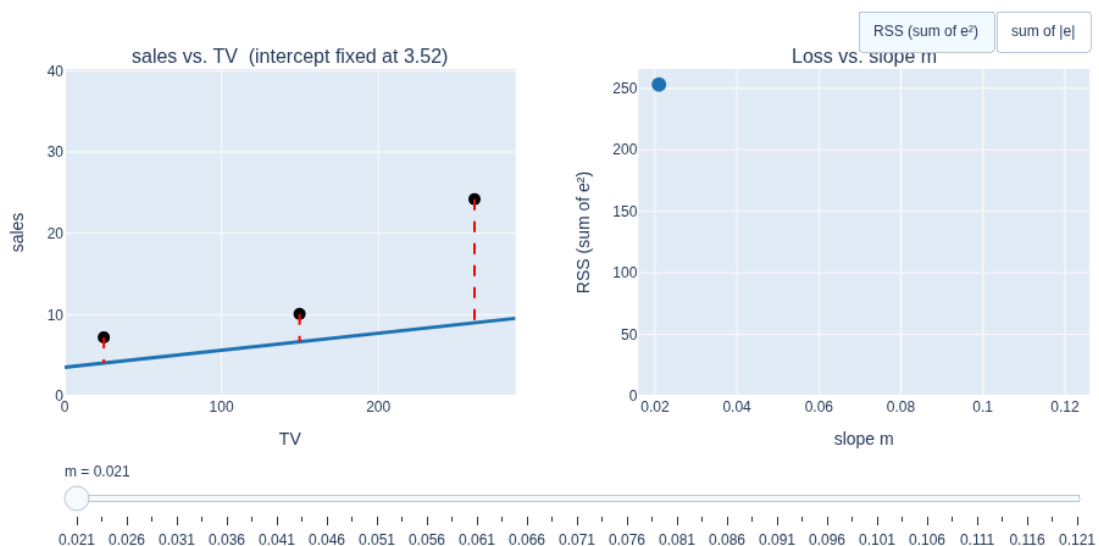


Discuss: Where is each loss smallest? Do RSS and sum of $|e|$ pick the same intercept? Compare the shapes: RSS is a smooth bowl, while sum of $|e|$ is made of straight pieces with corners.

Sweeping the slope

Now fix the intercept at Line C's value and slide the slope m instead. The line pivots around the point where it crosses the y -axis.

```
In [13]: m_values = np.linspace(m_best - 0.05, m_best + 0.05, 41) # 41 slopes centered on m_best
loss_explorer(pts, 'm', m_values, b=b_best)
```



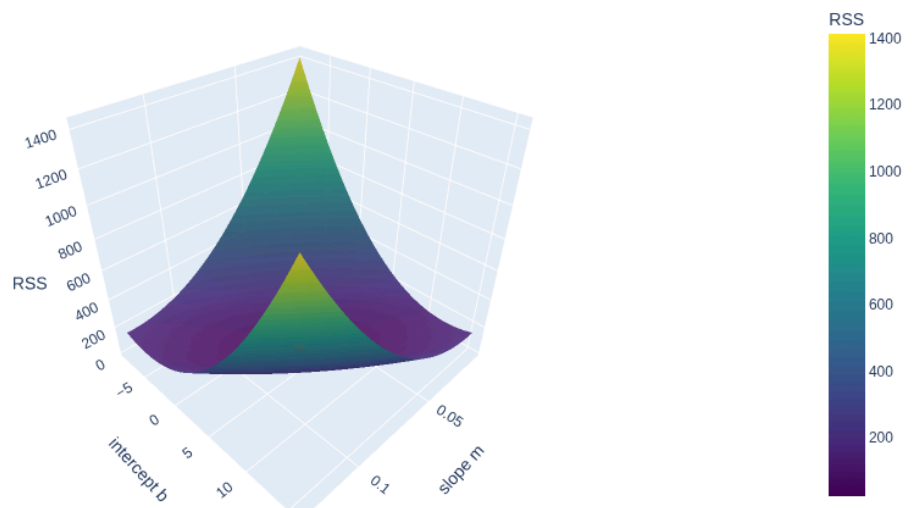
Finding the global minimum

So far we varied one parameter at a time, holding the other fixed. But RSS depends on m and b together: it's a surface over the (m, b) plane. Where is its lowest point?

```
In [14]: from regression_demos_helpers import rss_surface_3d, rss_contour

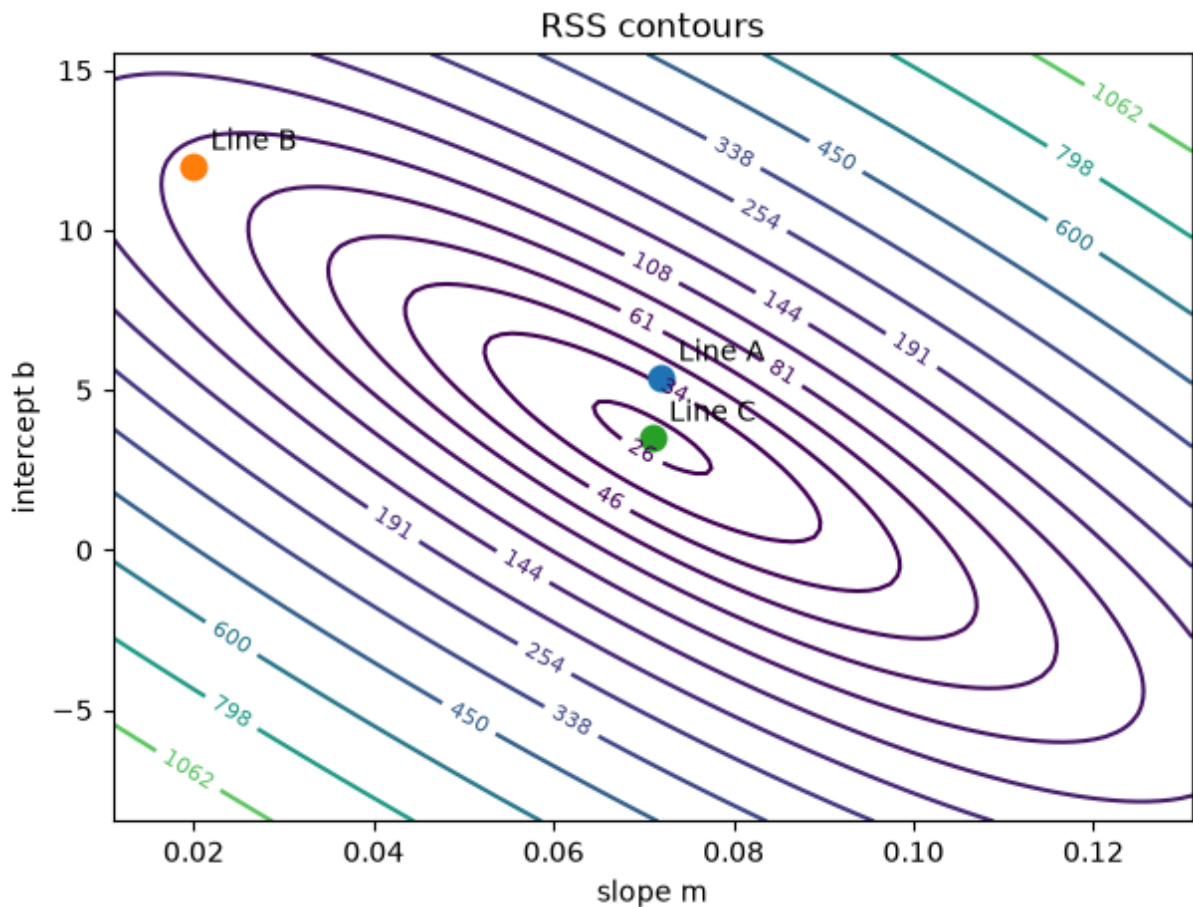
# A grid of (slope, intercept) pairs around Line C
m_grid = np.linspace(m_best - 0.06, m_best + 0.06, 101)
b_grid = np.linspace(b_best - 12, b_best + 12, 101)

# RSS for every pair on the grid; the red dot is the lowest point
rss_surface_3d(pts, m_grid, b_grid)
```



The same surface as a contour plot, like a topographic map: each curve connects (m, b) pairs with the same RSS. Every candidate line is a single point on this map.

```
In [15]: rss_contour(pts, m_grid, b_grid, lines); # Line D is far off this map
```



A formula for the best line

Using calculus (set both partial derivatives of RSS to zero; we'll skip the details), the minimum is at

$$\hat{m} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \quad \hat{b} = \bar{y} - \hat{m} \bar{x}$$

The book writes these as $\hat{\beta}_1$ (slope) and $\hat{\beta}_0$ (intercept). Hardly anyone has them memorized, and you don't need to. The point is that there's a *formula* for the best slope and intercept, so no searching is needed.

```
In [16]: x, y = pts['TV'], pts['sales']

m_hat = ((x - x.mean()) * (y - y.mean())).sum() / ((x - x.mean()) ** 2).sum()
b_hat = y.mean() - m_hat * x.mean()
print(f'slope = {m_hat:.4f}, intercept = {b_hat:.4f}')
```

```
slope = 0.0710, intercept = 3.5155
```

In practice, you call a library that does this for you. Here are a couple of examples:

```
In [17]: from sklearn.linear_model import LinearRegression

# NumPy: fit a degree-1 polynomial; returns [slope, intercept]
m_np, b_np = np.polyfit(x, y, deg=1)

# scikit-learn: X must be 2-D (a DataFrame with one column)
sk_model = LinearRegression().fit(pts[['TV']], y)

print(f'numpy:          slope = {m_np:.4f}, intercept = {b_np:.4f}')
print(f'scikit-learn: slope = {sk_model.coef_[0]:.4f}, intercept = {sk_model.intercept_:.4f}')

numpy:          slope = 0.0710, intercept = 3.5155
scikit-learn: slope = 0.0710, intercept = 3.5155
```

Should we spend more on TV advertising?

```
In [44]: # Use Line C, the least squares line through our 3 stores (m_hat, b_hat from

# Predict sales corresponding to current max spending on TV
x_max = Advertising['TV'].max()
y_max = m_hat * x_max + b_hat

# Predict sales for 100$ more than current max spending on TV
x_new = x_max + 100
y_new = m_hat * x_new + b_hat

# Compare the two predictions
print(f"Current sales: {y_max:.2f} units")
print(f"Predicted sales for 100k$ more in TV spending: {y_new:.2f} units$")

# Print percentage increase
percentage_increase = ((y_new - y_max) / y_max) * 100
print(f"Percentage increase: {percentage_increase:.2f}%")
```

Current sales: 24.55 units
 Predicted sales for 100k\$ more in TV spending: 31.65 units\$
 Percentage increase: 28.91%

Using a line based on a particular 3-store sample, an estimated 30 percent increase is impressive (depending on how much each unit costs, sells for), but **how confident** should we be in our prediction?

Formalizing Confidence

Three questions we'd like to answer about a fitted line:

1. **R^2** : How much better does the model predict the data than just predicting the mean? Is that enough to quantify confidence?
2. **p-value**: If there were really no relationship, how likely is it that a random sample would still produce an R^2 this big (or bigger)?

3. **Standard error:** How precisely did we estimate the slope and intercept?
Can we put error bars on them, or give a range of values we're 95% confident about?

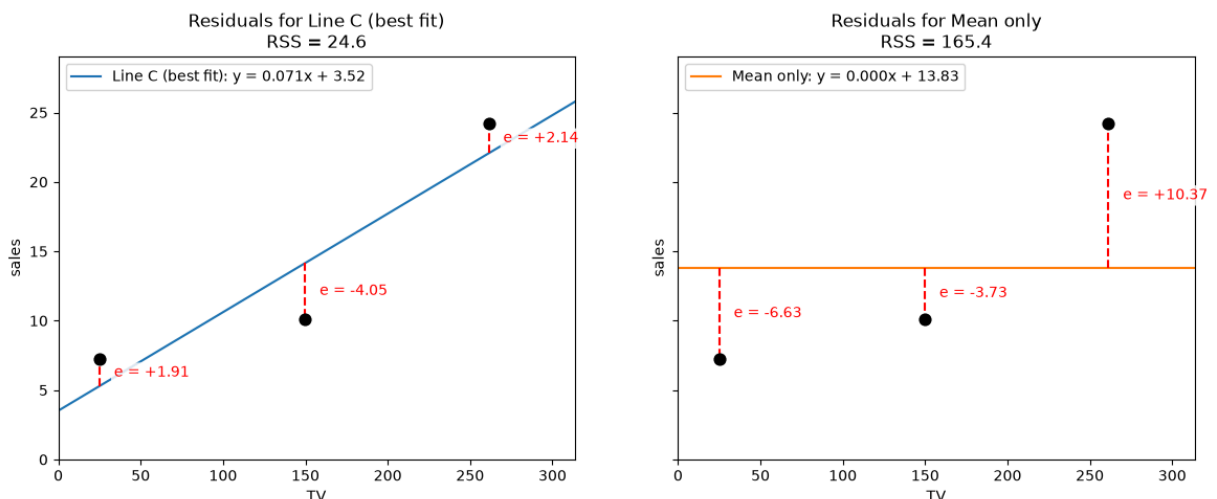
R^2 : better than guessing the mean?

The simplest possible model ignores **TV** and always predicts the average sales, $\bar{y}$. Compare its residuals to Line C's:

```
In [19]: from matplotlib.pyplot import subplots
from regression_demos_helpers import r_squared, plot_scenario

# Line C vs. a flat line at the mean of sales (slope 0)
compare = {'Line C (best fit)': (m_best, b_best),
           'Mean only': (0, pts['sales'].mean())}

fig, axes = subplots(1, 2, figsize=(14, 5), sharey=True)
plot_residuals(pts, compare, 'Line C (best fit)', ax=axes[0], show_rss=True)
plot_residuals(pts, compare, 'Mean only', ax=axes[1], show_rss=True);
```



How much better is the model than just predicting the average?

The mean-only line's sum of squares has its own name, the **total sum of squares** (TSS). It measures how much y varies in the first place. It's tempting to just subtract, TSS – RSS. But that number depends on the units of y :

```
In [20]: x, y = pts['TV'], pts['sales'] # sales is recorded in thousands of units

# Same data and same fit, with sales measured two ways
for scale, unit in [(1, 'thousands of units'), (1000, '(plane/ordinary) unit')]:
    y_scaled = y * scale
    m, b = fit_line(x, y_scaled)
    rss = np.sum((y_scaled - (m * x + b)) ** 2)
    tss = np.sum((y_scaled - y_scaled.mean()) ** 2)
    print(f'sales in {unit}: TSS - RSS = {tss - rss:>14,.1f}      '
          f'(TSS - RSS) / TSS = {(tss - rss) / tss:.3f}')
```

sales in thousands of units: $TSS - RSS = 140.8 \quad (TSS - RSS) / TSS = 0.851$

sales in (plane/ordinary) units: $TSS - RSS = 140,801,211.8 \quad (TSS - RSS) / TSS = 0.851$

Changing units multiplies the difference by a million, even though the fit is exactly as good. The *fraction* doesn't change, and that fraction is R^2 :

$$R^2 = \frac{TSS - RSS}{TSS} = 1 - \frac{RSS}{TSS}$$

It's the fraction of the variation in y that the model explains (the "variance explained").

▼ What R-squared value indicates a perfect fit? And what about a terrible fit?

Perfect fit: $RSS = 0$ and $TSS > 0$ plug into equation for R^2 , and get $R^2 = 1$.

In practice, for a great fit, we'll have RSS , slightly greater than 0 and $TSS > 0$ so, R^2 will be slightly less than 1.

Terrible fit: $RSS \approx TSS$ (The model predicts y -values no better than $\bar{y}$) plug into equation for R^2 , and get $R^2 = 0$.

In practice, terrible fits will typically have R^2 values slightly greater than 0.

▼ Terminology Gotcha:

Given that, $R^2 = 1 - \frac{RSS}{TSS}$

...then is this true?

$$R = \sqrt{1 - \frac{RSS}{TSS}} < - \text{This is NOT true!!!}$$

The answer is **No!**

" R^2 " is *not* a mathematical expression; It is just a *name*. We refer to the statistic above as "R-squared" and we often write that name like this: " R^2 ". This confusing (and I think *terrible*) name is an unfortunate accident of history and for now, we're stuck with it.

▼ R-Squared as Fraction of Variance Explained

Purpose:

Intuitively, the y values of our data have some kind of average spread or variation (the variance in y). Our *model* can account for or "explain" a certain amount of that spread or variation. R^2 is a measure of how well (or poorly) our model "explains" that variation.

Variance measures how spread out a set of numbers is: the average squared distance from their mean.

$$\text{var}(z) = \frac{1}{n} \sum_{i=1}^n (z_i - \bar{z})^2$$

Step 1: split the fraction.

$$R^2 = \frac{\text{TSS} - \text{RSS}}{\text{TSS}} = \frac{\text{TSS}}{\text{TSS}} - \frac{\text{RSS}}{\text{TSS}}$$

Step 2: divide each numerator and denominator by n , the number of residuals. This doesn't change either fraction.

$$R^2 = \frac{\text{TSS}/n}{\text{TSS}/n} - \frac{\text{RSS}/n}{\text{TSS}/n}$$

Step 3: turn each $(\dots)/n$ into a variance. We have two sets of residuals:

- Predicting with $\bar{y}$: $e_i^{\bar{y}} = y_i - \bar{y}$, so $\text{TSS} = \sum_i (e_i^{\bar{y}})^2$
- Predicting with the model: $e_i^{\text{model}} = y_i - \hat{y}_i$, so $\text{RSS} = \sum_i (e_i^{\text{model}})^2$

Both sets of residuals have a mean of 0 (for the least squares line, that's the $\sum_i e_i = 0$ fact from earlier). With $\bar{e} = 0$, a sum of squared residuals divided by n is exactly their variance:

$$\frac{\text{TSS}}{n} = \frac{1}{n} \sum_i (e_i^{\bar{y}} - 0)^2 = \text{var}(e^{\bar{y}}) \qquad \frac{\text{RSS}}{n} = \text{var}(e^{\text{model}})$$

Substituting:

$$R^2 = \frac{\text{var}(e^{\bar{y}})}{\text{var}(e^{\bar{y}})} - \frac{\text{var}(e^{\text{model}})}{\text{var}(e^{\bar{y}})}$$

The first term is the whole. $\text{var}(e^{\bar{y}})$ is all of the variance in the residuals when we make the naive prediction $\bar{y}$. Divided by itself, it's 1: 100% of the variance in $e^{\bar{y}}$. Notice that $\text{var}(e^{\bar{y}})$ is also just $\text{var}(y)$ by the definition of variance.

The second term is the fraction left over. It's the fraction of that whole (the variance in y) that still remains in the residuals when we compute residuals using the model.

- If $\frac{\text{var}(e^{\text{model}})}{\text{var}(e^{\bar{y}})} = 0.01$, the model fits the data very well: its residuals have only 1% of the variance of the $\bar{y}$ residuals. Then $R^2 = 0.99$.

- If $\frac{\text{var}(e^{\text{model}})}{\text{var}(e^{\bar{y}})} \approx 0.9$, computing the residuals with the model gives a spread very similar to computing them with $\bar{y}$. The model barely beats guessing the mean, and $R^2 \approx 0.1$.

Putting it together:

$$R^2 = 1 - \frac{\text{var}(e^{\text{model}})}{\text{var}(e^{\bar{y}})}$$

is the fraction by which the model *reduced* the variance in the residuals, relative to the naive prediction $\bar{y}$. So, R^2 can be interpreted as variance in y explained by the model. In the jargon of statistics (and in ISL, our textbook), this is the fraction of **variance explained** by the model.

Conclusion: Intuitively, the y values have some kind of average spread or variation (the variance in y). Our *model* can account for or "explain" a certain amount of that spread or variation. R^2 is a measure of how well (or poorly) our model "explains" that variation.

```
In [21]: print(f'R2 for Line C on our three stores: {r_squared(x, y):.3f}')
```

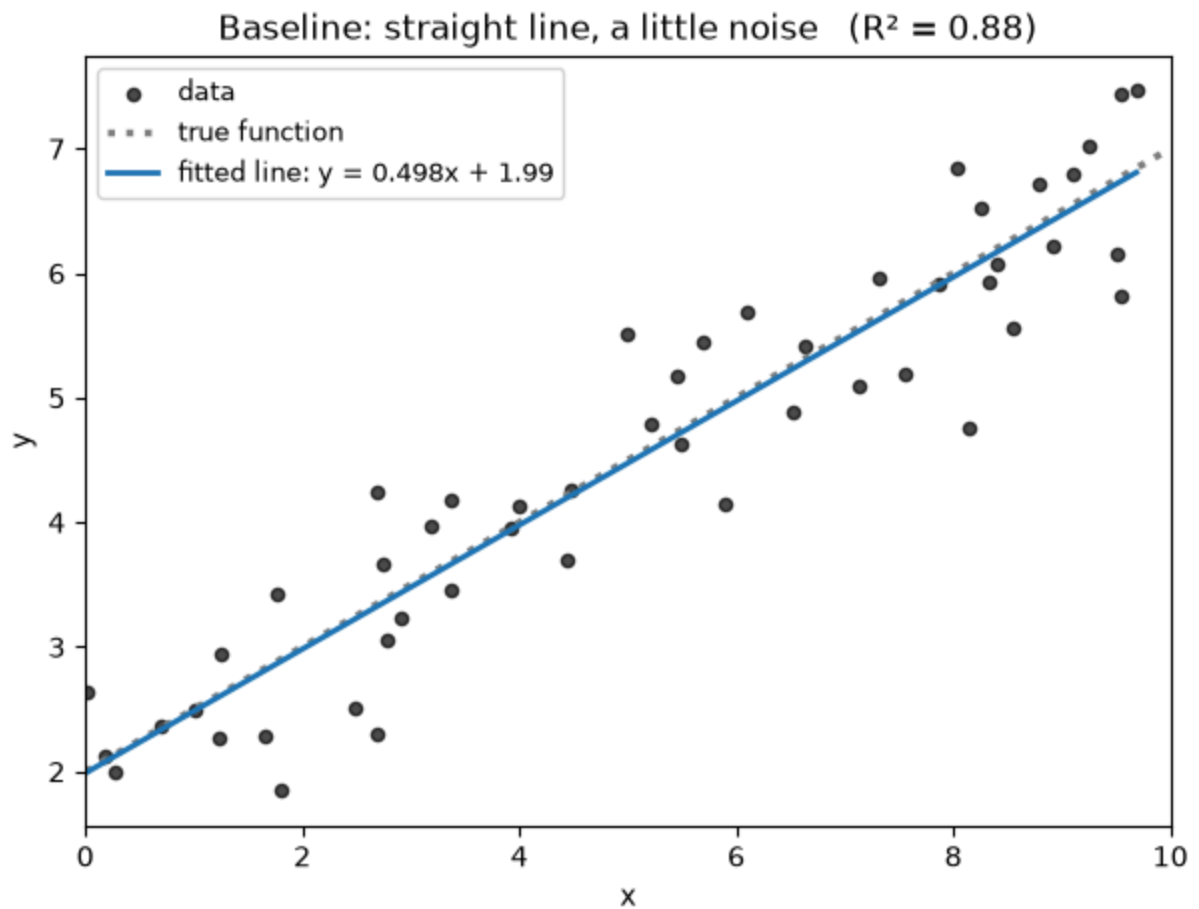
R² for Line C on our three stores: 0.851

A Few Cases

We're leaving the Advertising data now. The next examples use simulated data, so we know the true function behind each one (the dotted gray curve). First, a baseline where a straight line fits well:

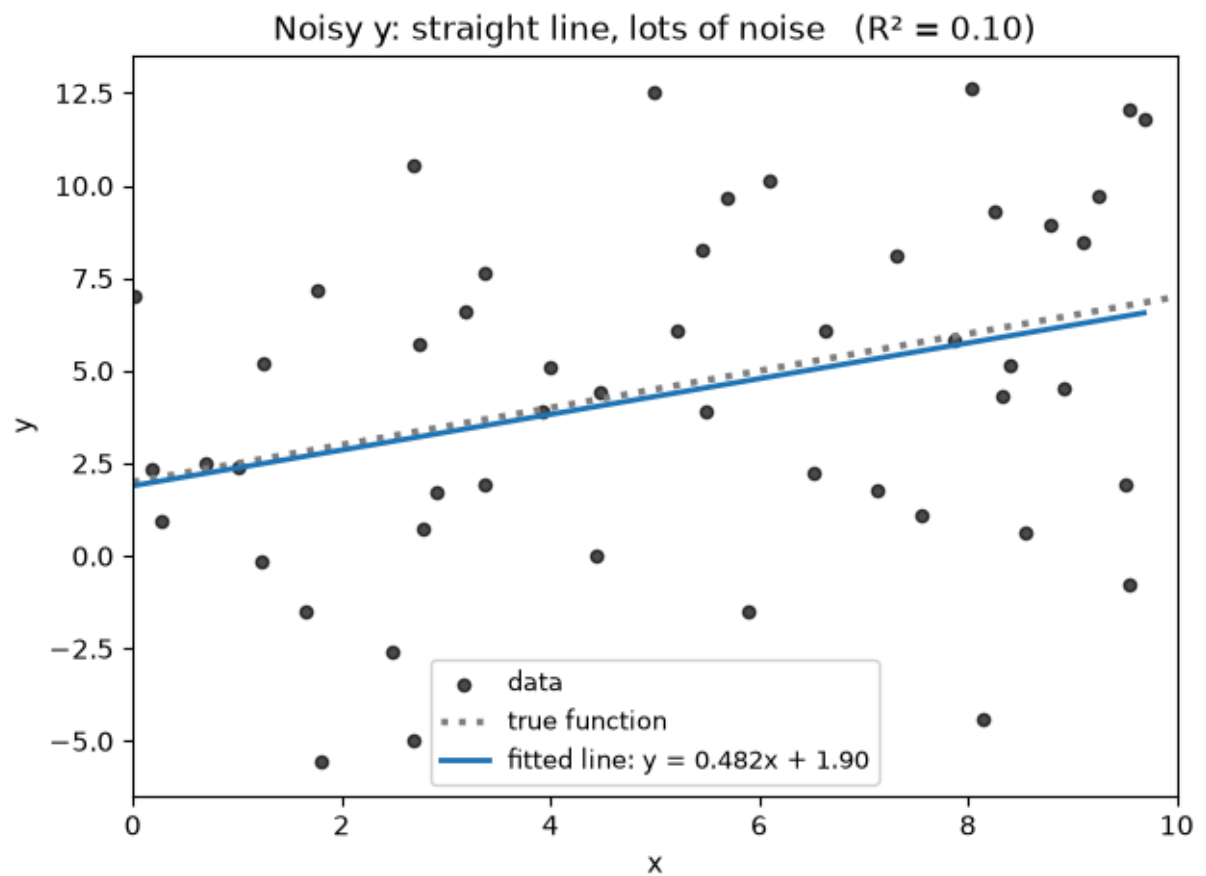
A **pretty good fit** to the sample, so a pretty good R^2

```
In [22]: # Simulated data (not Advertising), generated with a fixed seed in regressio
plot_scenario('baseline');
```



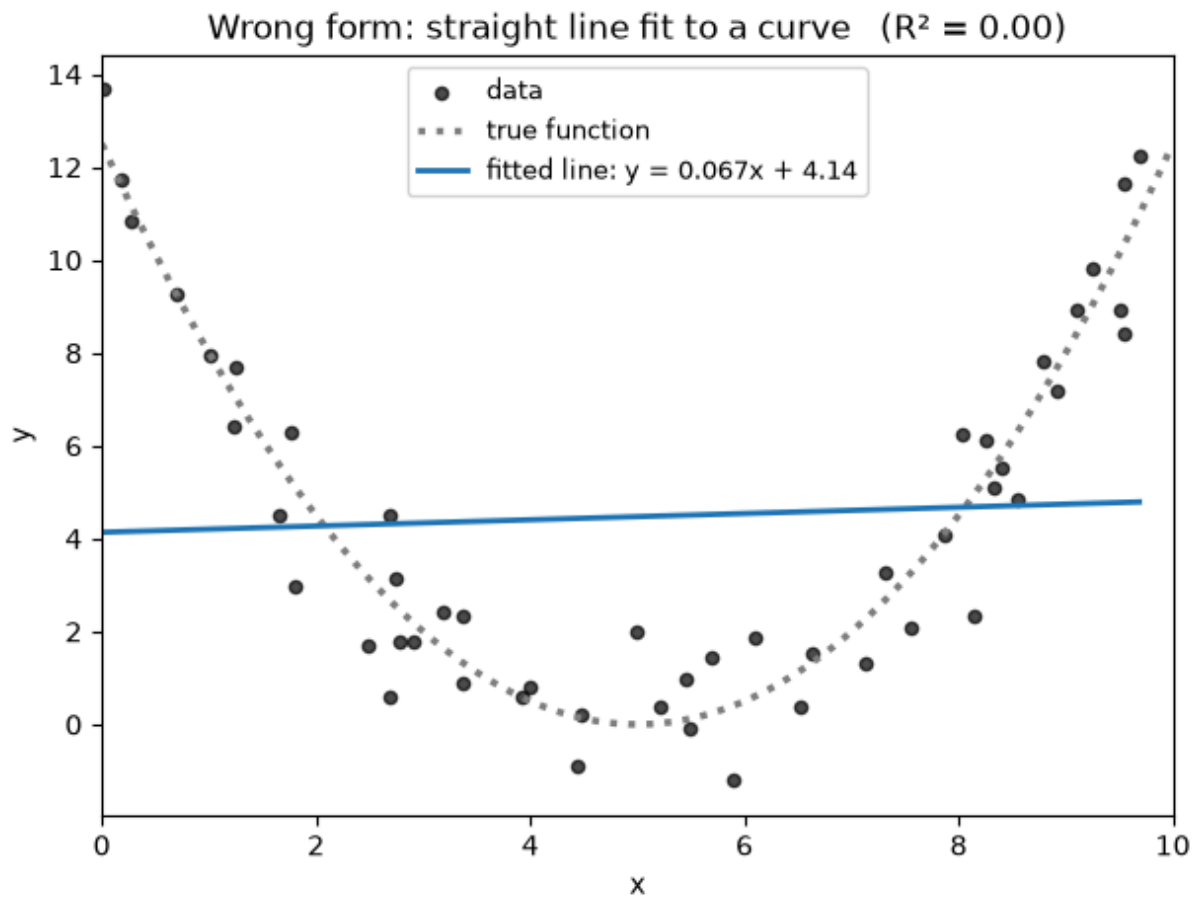
Noisy y . Same true line, but with much more variation that x can't explain. The fitted line is still close to the truth, yet R^2 is low.

```
In [23]: plot_scenario('noisy');
```



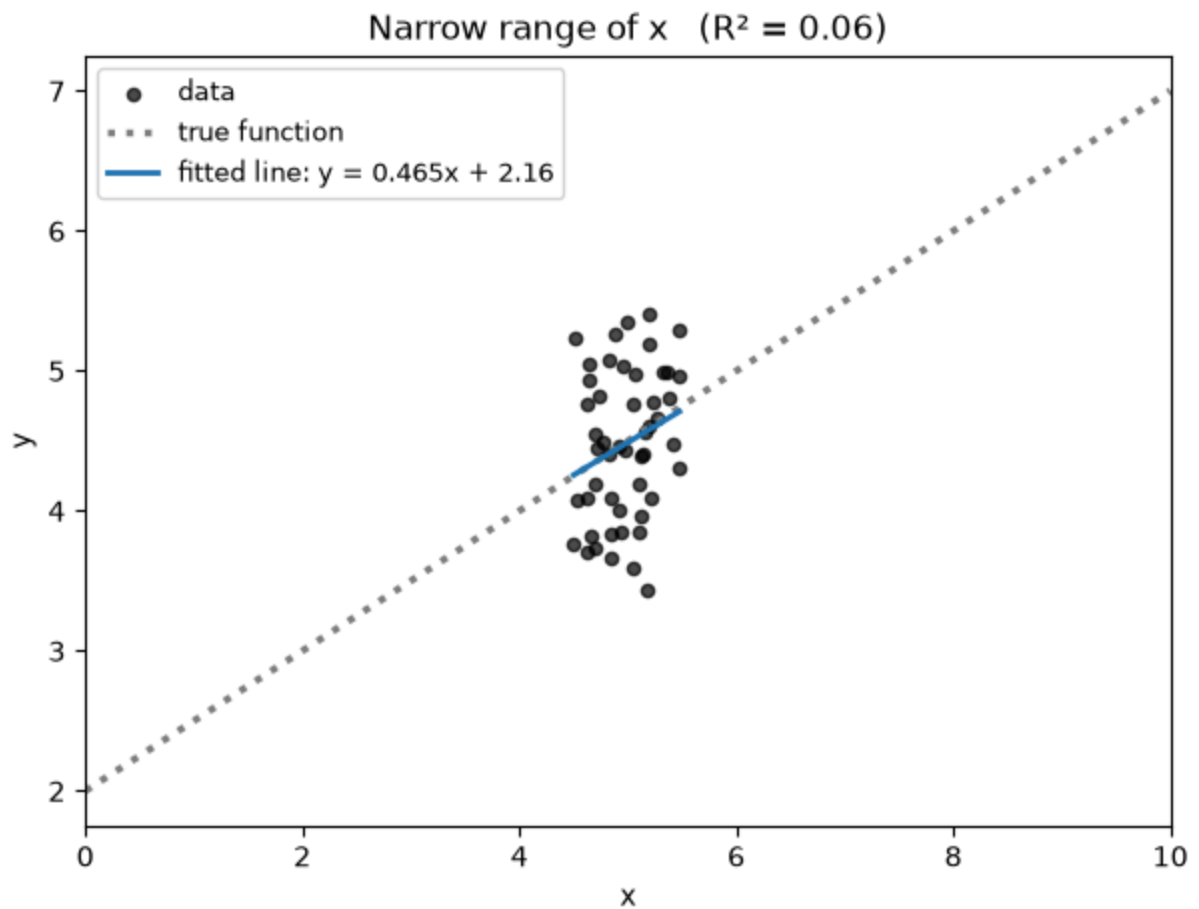
Wrong functional form. The truth is a curve (a parabola plus a little noise). No straight line can follow it.

```
In [24]: plot_scenario('curved');
```



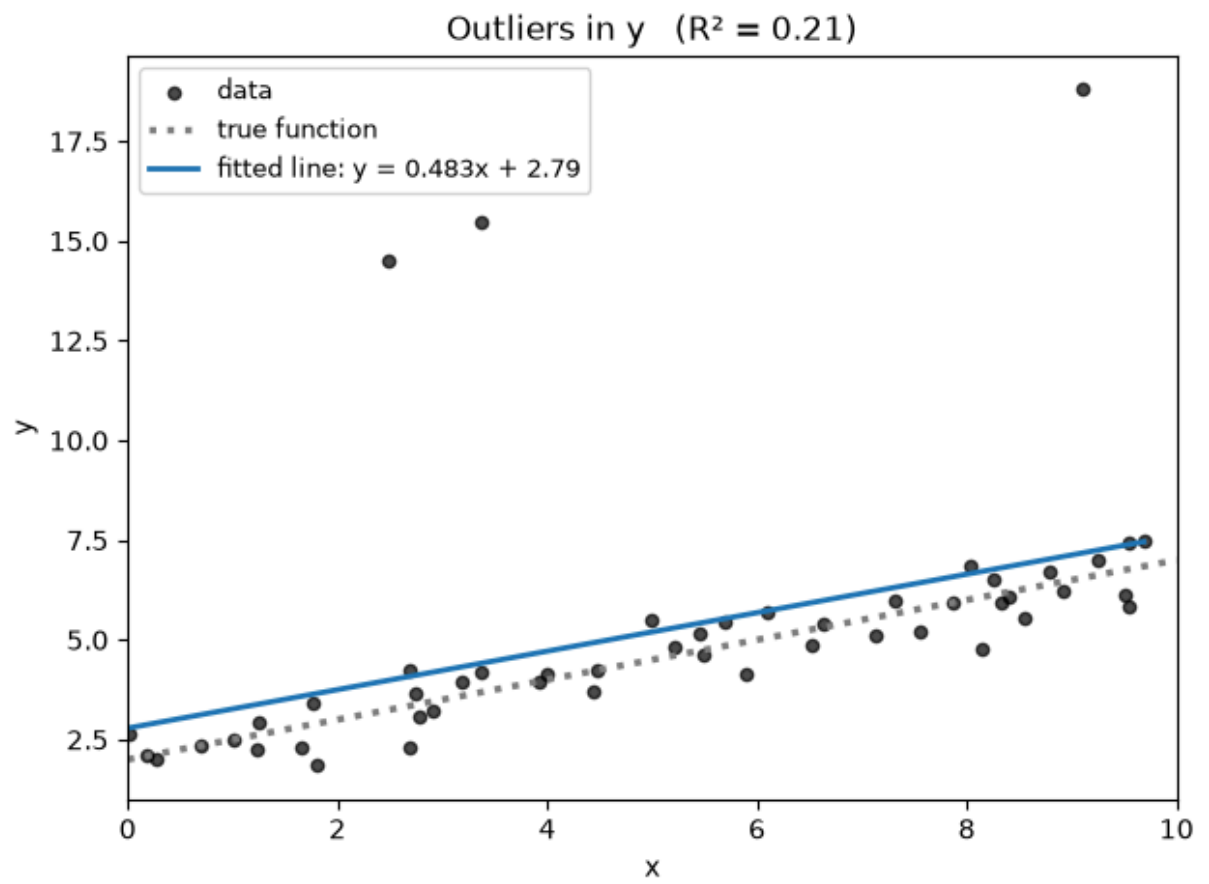
Narrow range of x . Same true line and noise level as the baseline, but every x falls between 4.5 and 5.5. Over such a short range, the noise swamps the trend.

```
In [25]: plot_scenario('narrow');
```



Outliers in y . The baseline again, with three points pushed far upward. A few extreme points inflate RSS and translate or tilt the line.

```
In [26]: plot_scenario('outliers');
```



Section 2

p-Value

Is R^2 enough to quantify confidence?

In the below plots, pretend we don't have all the data. We draw a sample of 3 points and fit a line to each sample set.

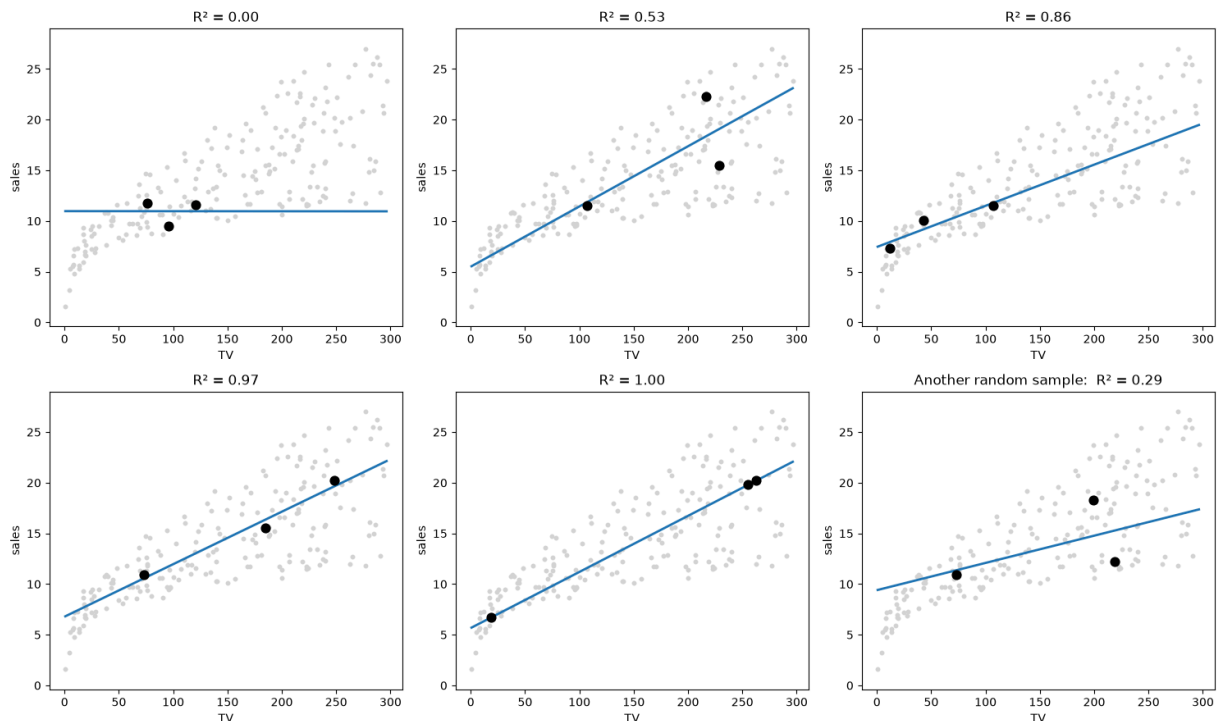
▼ Answer:

No. R^2 depends on the sample.

- A low R^2 can mean a real but noisy relationship, the wrong shape of model, too narrow a range of data, or a few outliers.
- And a high R^2 from only three points (the beginning of this section), could just be due to a "unlucky" sample. Unlucky, because if we rely on R^2 inappropriately, we have false confidence in our model's ability to make predictions, when really, it just happens to do a good job making predictions on the data *we happened to pick*. The next two topics (p-values and standard errors) address this.

```
In [27]: # Back to the real Advertising data: draw 1000 random samples of 3 stores at
# Shown: five of those samples, from the lowest  $R^2$  to the highest, then one
from regression_demos_helpers import plot_sampleLuck

plot_sampleLuck(Advertising, n=3, draws=1000);
```

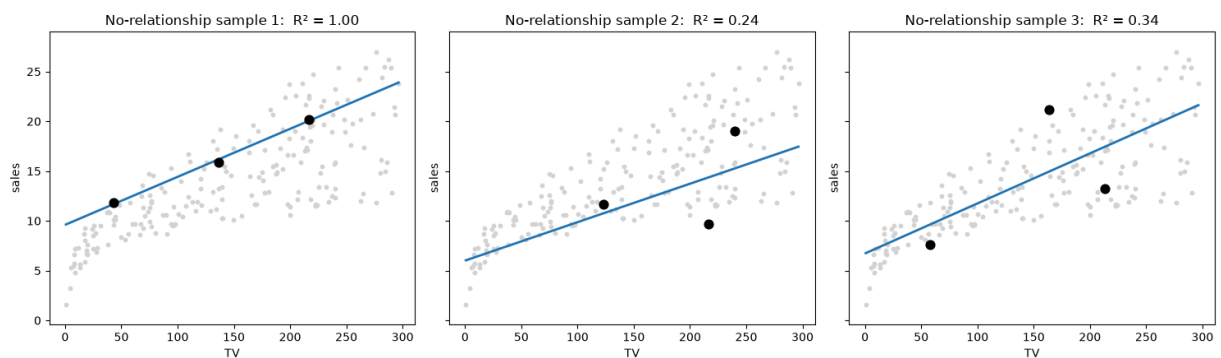


Line C is the least squares line through our three stores, with $R^2 \approx 0.85$. That looks good, but three points is very little data. **If TV had no relationship to sales at all, how often would three random stores give an R^2 at least this large just by luck?**

Breaking the relationship on purpose. To see what luck alone can do, we build samples where `TV` and `sales` have nothing to do with each other: take the TV budgets of 3 random stores and the sales of 3 *other* random stores, and pair them up. Any pattern in such a sample is pure coincidence.

```
In [28]: # Real Advertising values, but each sample pairs TV from 3 random stores with
from regression_demos_helpers import plot_null_samples

plot_null_samples(Advertising, n=3, k=3);
```

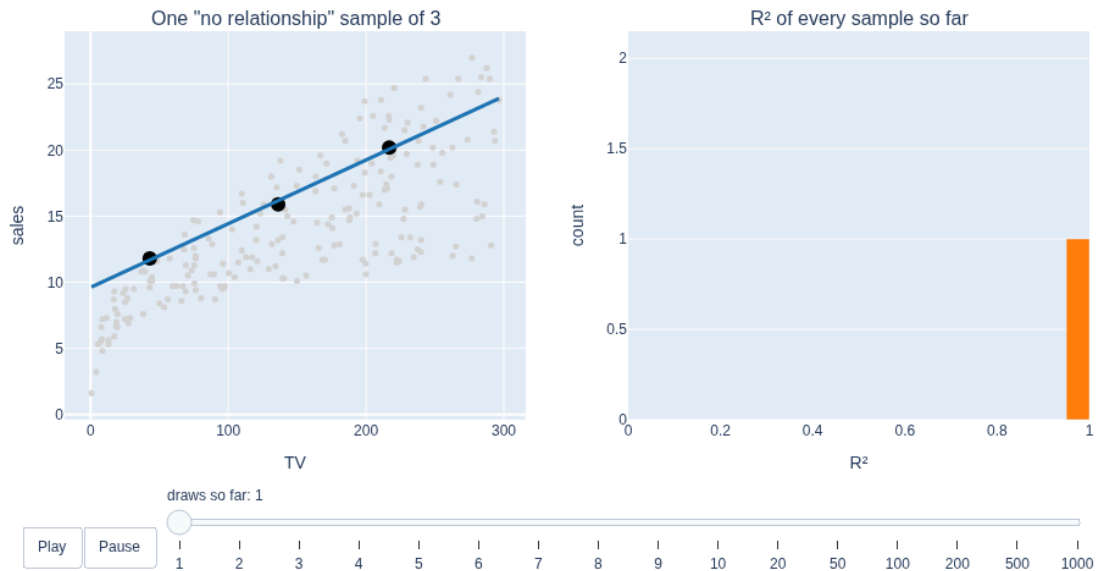


Even with no relationship, some samples produce a steep line and a high R^2 . How often? Let's draw 1000 of these samples and build up a histogram of their R^2 values. Press **Play** or drag the slider: the first 10 draws are added one at a time, then in bigger jumps.

```
In [29]: from regression_demos_helpers import null_r2_explorer

null_r2_explorer(Advertising, n=3, draws=1000)
```

Draw 1: $R^2 = 1.00$ (orange bar) Histogram of all 1 draws so far



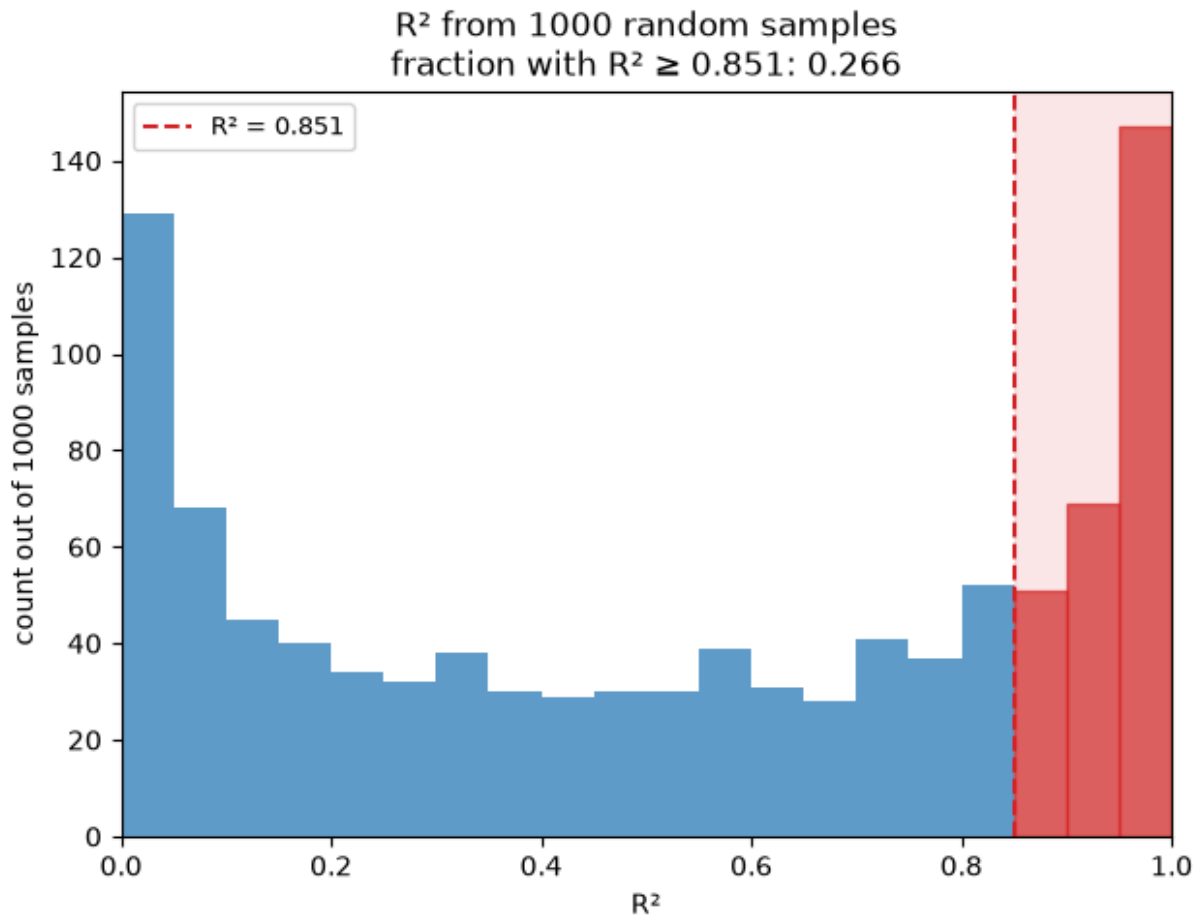
Now compare Line C against this pile of luck-only values. What fraction of the "no relationship" samples did at least as well as Line C?

```
In [30]: from regression_demos_helpers import null_r2, plot_r2_histogram

r2_C = r_squared(pts['TV'], pts['sales'])
_, null_values = null_r2(Advertising, n=3, draws=1000) # same 1000 draws as
p_simulated = np.mean(null_values >= r2_C)

plot_r2_histogram(null_values, shade_above=r2_C)
print(f'Fraction of no-relationship samples with  $R^2 \geq \{r2\_C:.3f\}$ : {p_simulated}')
```

Fraction of no-relationship samples with $R^2 \geq 0.851$: 0.266



The p-value

That fraction (0.266 in the plot) is called the **p-value**: *if there were no relationship, between TV and Sales, how often would random chance give an R^2 at least as good as our model's R^2 ?*

Here, roughly a quarter of the "no relationship" samples of 3 stores did as well as Line C. Despite its "good" R^2 , Line C is not convincing evidence that TV spending is related to sales.

- A **small** p-value (a common cutoff is 0.05) means our result would be very surprising if there were no relationship, so we have evidence that there is one.
- The p-value is *not* the probability that the model is wrong, nor the probability that there's no relationship. It only measures how surprising a fit this good (an R^2 at least this large) would be if there were no relationship.

In practice, nobody simulates this by hand. Libraries compute the p-value from a formula (which assumes the residuals are roughly normally distributed), so expect a close match, not an identical one:

```
In [31]: from scipy.stats import linregress

# fit_line (and np.polyfit) only return the slope and intercept, so refit with
fit_C = linregress(pts['TV'], pts['sales'])
print(f'slope: {fit_C.slope:.4f} (Line C: {m_best:.4f})')
print(f'simulated p-value: {p_simulated:.3f}')
print(f'scipy p-value:      {fit_C.pvalue:.3f}')

slope: 0.0710 (Line C: 0.0710)
simulated p-value: 0.266
scipy p-value:      0.252
```

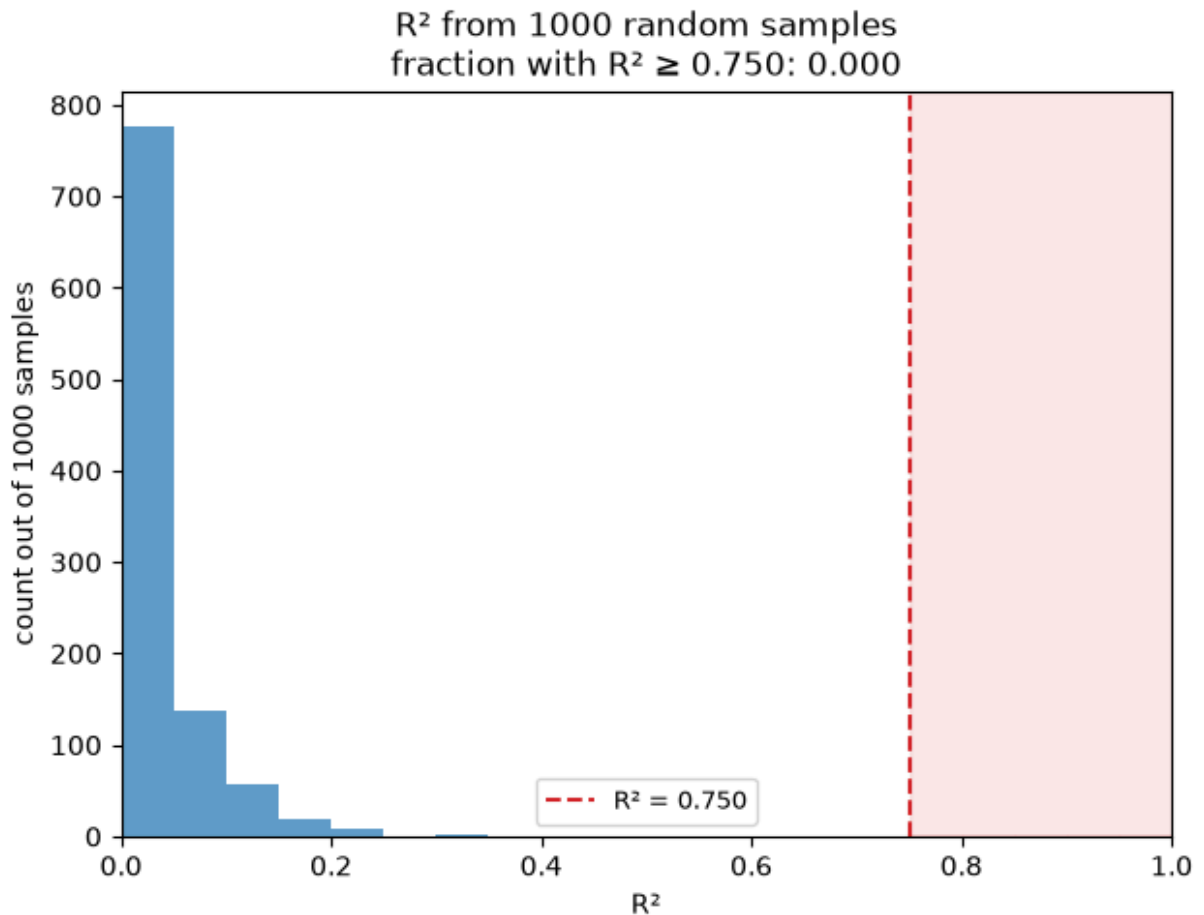
What if we had more data?

Repeat the whole experiment with 30 stores instead of 3: take a real random sample of 30 stores and fit a line, then compare its R^2 to 1000 "no relationship" samples of 30.

```
In [32]: stores_30 = Advertising.sample(30, random_state=463) # a real sample: TV a
r2_30 = r_squared(stores_30['TV'], stores_30['sales'])
_, null_30 = null_r2(Advertising, n=30, draws=1000)
fit_30 = linregress(stores_30['TV'], stores_30['sales'])

plot_r2_histogram(null_30, shade_above=r2_30)
print(f'R² for 30 real stores: {r2_30:.3f}')
print(f'No-relationship samples at least that good: {np.sum(null_30 >= r2_30)}')
print(f'scipy p-value: {fit_30.pvalue:.2g}')

R² for 30 real stores: 0.750
No-relationship samples at least that good: 0 of 1000
scipy p-value: 6.2e-10
```



Remember, the p-value is the blue area to the right of the vertical dashed line divided by the total blue area. That's basically $\text{p-value} = 0/(\text{a big number}) = 0$.

With 30 random, no-relationship samples:

- Luck alone almost never produces a large R^2
- The "no relationship" histogram piles R^2 values near 0
- The real sample's R^2 sits far out in the tail (vertical red, dashed line).
- None of the 1000 simulated samples reach $R^2 = 0.75$, so the p-value is essentially 0. **The same kind of fit is far more convincing with more data.**

Standard error

The p-value asked whether there's a relationship at all. The third question asks **how precisely** we've pinned down the slope (or other model parameters): if we collected a new sample, how much would our estimate be likely to change?

We're leaving the real data again. To measure how much the slope estimate bounces from sample to sample, we need to know the true line the samples come from, so in this next section we simulate noisy data so we have the formula for that true line.

▼ How this simulation works (optional)

- **A pretend true line.** With real data we never know the true line, so we *treat* the least squares line through all 200 Advertising stores as though it were the truth.
- **Noise of a realistic size.** Each simulated store's sales are the true line's prediction plus fresh random noise (centered on 0). The noise's typical size matches how far the real stores sit from that line (their residual standard error, printed below as `noise sd`). We don't reuse the real residuals, just their spread.
- **Same TV budgets every time.** Every simulated sample uses the same 30 TV budgets; only the noise (and so the sales) changes.
- **A simplification.** The simulated noise has the same spread at every TV value, while the real residuals fan out as TV grows. That doesn't matter for this demo.

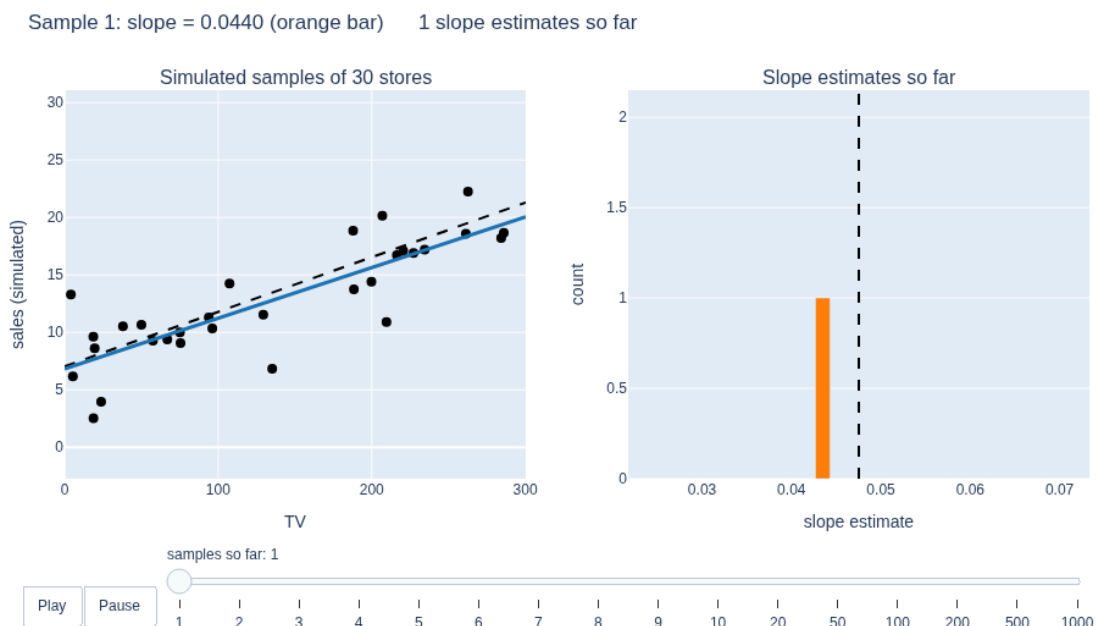
Press **Play**: each new sample gives a slightly different fitted line and a slightly different slope.

```
In [33]: # Simulated data: a known true line, with its numbers borrowed from all 200
from regression_demos_helpers import make_truth, slope_explorer

truth = make_truth(Advertising, n=30) # true slope and intercept, noise sd
print(f"true line: sales = {truth['m']:.4f} TV + {truth['b']:.2f},    noise sd = {truth['sd']:.2f}")

slope_explorer(truth, draws=1000)
```

true line: sales = 0.0475 TV + 7.03, noise sd = 3.26



The **standard error (SE)** of the slope is the typical size of this bounce: the standard deviation of that histogram. It tells you how much your estimate would change if you collected a new sample.

Notice the histogram is centered on the true slope: the estimates are right *on average*, but any single one typically misses by about one SE.

In real life, you have one sample, not 1000. But the software can estimate the bounce from that one sample alone (scipy calls it `stderr`; regression tables label it `std err`). Compare the SE from just the first simulated sample (orange bar) with the SD of all 1000 estimates (shaded band):

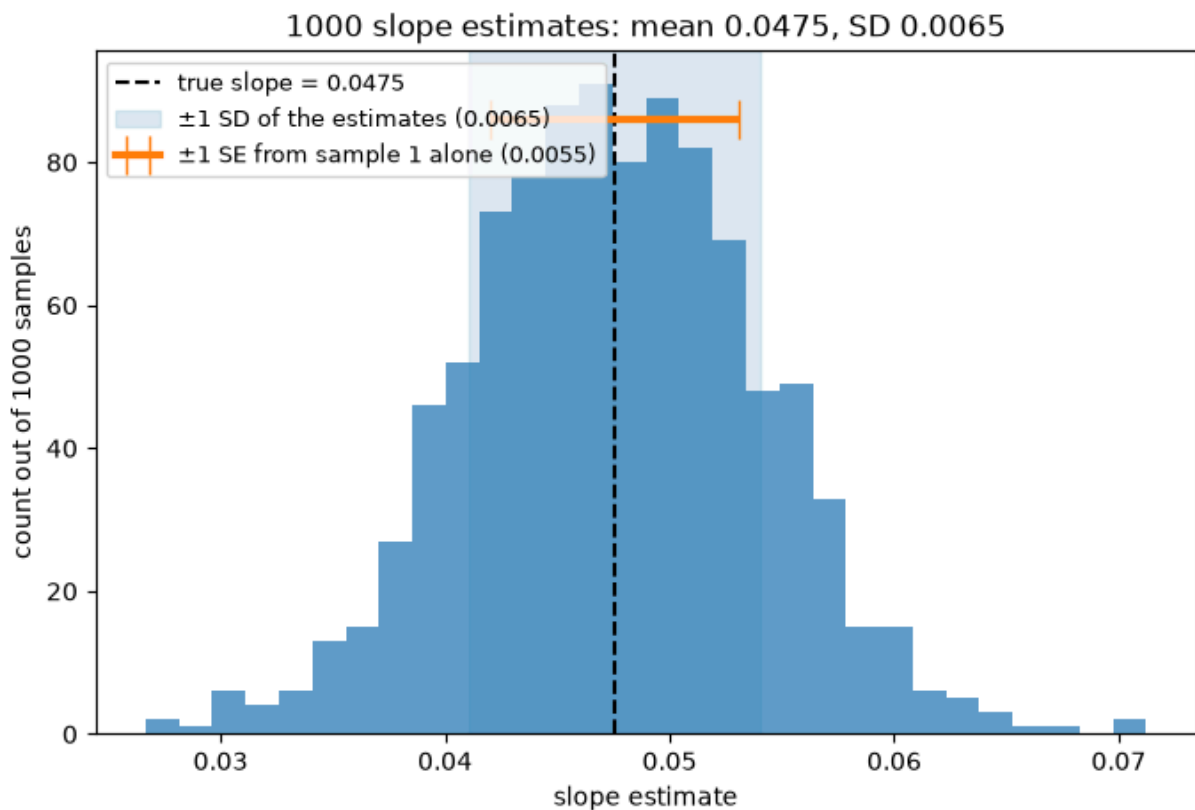
```
In [34]: from regression_demos_helpers import simulate_fits, plot_slope_histogram

ys, fits = simulate_fits(truth, draws=1000)  # the same 1000 samples as the
one_se = fits['stderr'][0]

plot_slope_histogram(fits['slope'], truth['m'], one_se=one_se)
print(f"SD of 1000 slope estimates (needs the simulation): {fits['slope'].std}")
print(f"SE from sample 1 alone (scipy's stderr): {one_se:.4f}")
```

SD of 1000 slope estimates (needs the simulation): 0.0065

SE from sample 1 alone (scipy's stderr): 0.0055



Close, from a single sample and no simulation. (Not identical: the SE is itself estimated from the sample, so it bounces a little too.)

Using the standard error

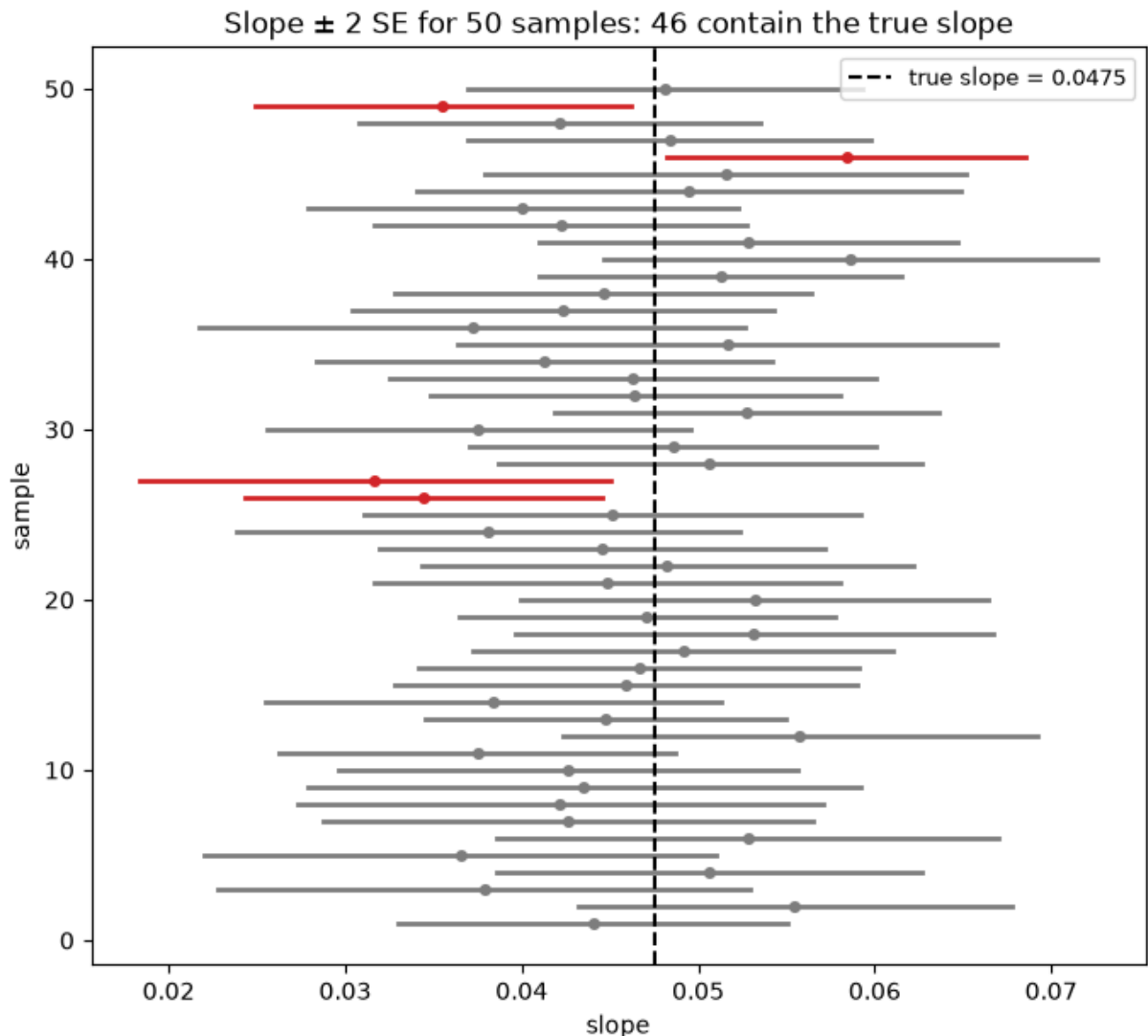
Confidence interval. Estimate ± 2 SE gives a range of plausible values for the true slope. Does it work? Here are the intervals from the first 50 simulated samples:

```
In [35]: from regression_demos_helpers import plot_intervals

plot_intervals(fits, truth['m'], k=50)

low, high = fits['slope'] - 2 * fits['stderr'], fits['slope'] + 2 * fits['st
covered = ((low <= truth['m']) & (truth['m'] <= high)).mean()
print(f'Fraction of all 1000 intervals that contain the true slope: {covered
```

Fraction of all 1000 intervals that contain the true slope: 0.943



About 95% of the intervals contain the true slope, so estimate ± 2 SE is called a **95% confidence interval**. (Any single interval either contains the truth or doesn't; the 95% describes how often the *method* works.)

t-statistic. Estimate $\div$ SE counts how many "bounces" the estimate is from zero. If it's far (say, more than 2), a true slope of zero is implausible. This is how

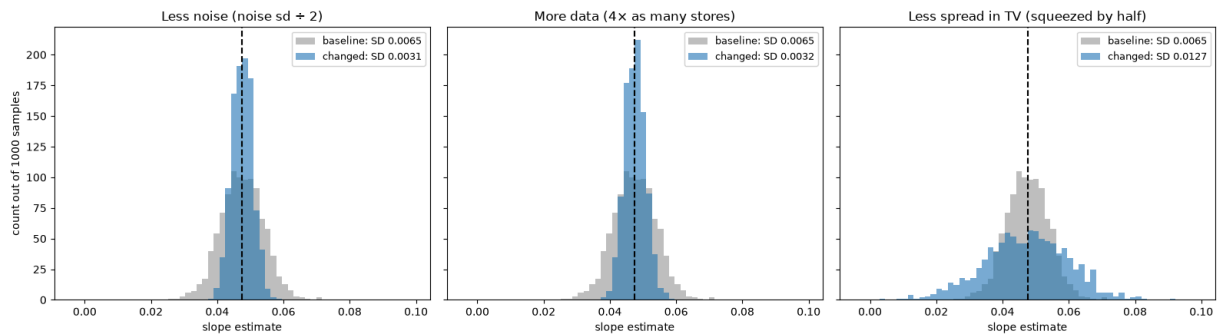
software computes the p-value from the previous section: a big t means a small p-value.

▼ Optional: what makes the SE smaller?

Less noise, more data, and more spread in x . Each panel below compares the baseline with one change. Halving the noise or using 4x as many stores cuts the SE in half; squeezing the TV budgets to half their spread doubles it.

```
In [36]: # Optional: still simulated, from the same true line
from regression_demos_helpers import plot_se_factors

plot_se_factors(truth);
```



Reading a regression table

Back to the real data, all 200 stores. This is the kind of table regression software prints (compare ISL Table 3.1):

```
In [37]: from scipy.stats import linregress

fit_all = linregress(Advertising['TV'], Advertising['sales'])
table = pd.DataFrame({'coef': [fit_all.intercept, fit_all.slope],
                      'std err': [fit_all.intercept_stderr, fit_all.stderr],
                      index=['intercept', 'TV']})
table['t'] = table['coef'] / table['std err']
table['95% CI low'] = table['coef'] - 2 * table['std err']
table['95% CI high'] = table['coef'] + 2 * table['std err']
table.round(4)
```

```
Out[37]:
```

	coef	std err	t	95% CI low	95% CI high
intercept	7.0326	0.4578	15.3603	6.1169	7.9483
TV	0.0475	0.0027	17.6676	0.0422	0.0529

In the table above, "intercept" refers to the model's y-intercept and **TV** refers to the model's estimate of the coefficient of the **TV** variable. In other words, **TV** refers to the slope of our linear model.

How to read the `TV` row: **the slope is 0.0475 with standard error 0.0027, so the true slope is probably between about 0.042 and 0.053, and it's clearly not zero** ($t \approx 17.7$: the estimate is almost 18 standard errors away from zero, so the p-value is essentially 0).

Putting it All Together: Should We Increase TV Advertising?

Back to the boss's questions from the very top of the notebook:

- "Should we increase TV advertising by \$100?"
- "How confident are you?"

Units first. In this dataset, `TV` is in thousands of dollars and `sales` is in thousands of units. So "+100" in the data means **+\$100,000** of TV advertising, and a slope of 0.0475 means about 47.5 more units sold per extra \$1,000. (ISL describes each row as a *market*; we've been calling them stores.)

The increase is slope × change. For a straight line, raising TV by 100 raises predicted sales by slope × 100, no matter what budget we start from; the intercept cancels out. So an interval for the slope turns into an interval for the increase: just multiply by 100.

Here's the answer from our 3 stores (the earlier "30% increase") next to the answer from all 200 markets:

```
In [38]: from scipy.stats import linregress, t as t_dist

extra_tv = 100    # TV is in thousands of dollars, so this is +$100,000

rows = {}
for label, data in [('3 stores (Line C)', pts), ('all 200 markets', Advertis):
    fit = linregress(data['TV'], data['sales'])
    multiplier = t_dist.ppf(0.975, len(data) - 2)    # how many SEs a 95% int
    rows[label] = {'slope': fit.slope, 'SE': fit.stderr, 'p-value': fit.pval,
                  'SEs for 95%': multiplier,
                  'increase (thousand units)': extra_tv * fit.slope,
                  'low': extra_tv * (fit.slope - multiplier * fit.stderr),
                  'high': extra_tv * (fit.slope + multiplier * fit.stderr)}

comparison = pd.DataFrame(rows).T
comparison.round(4)
```

Out[38]:

	slope	SE	p-value	SEs for 95%	increase (thousand units)	low	high
3 stores (Line C)	0.0710	0.0297	0.2521	12.7062	7.0978	-30.6032	44.7989
all 200 markets	0.0475	0.0027	0.0000	1.9720	4.7537	4.2231	5.2843

Same method, very different confidence:

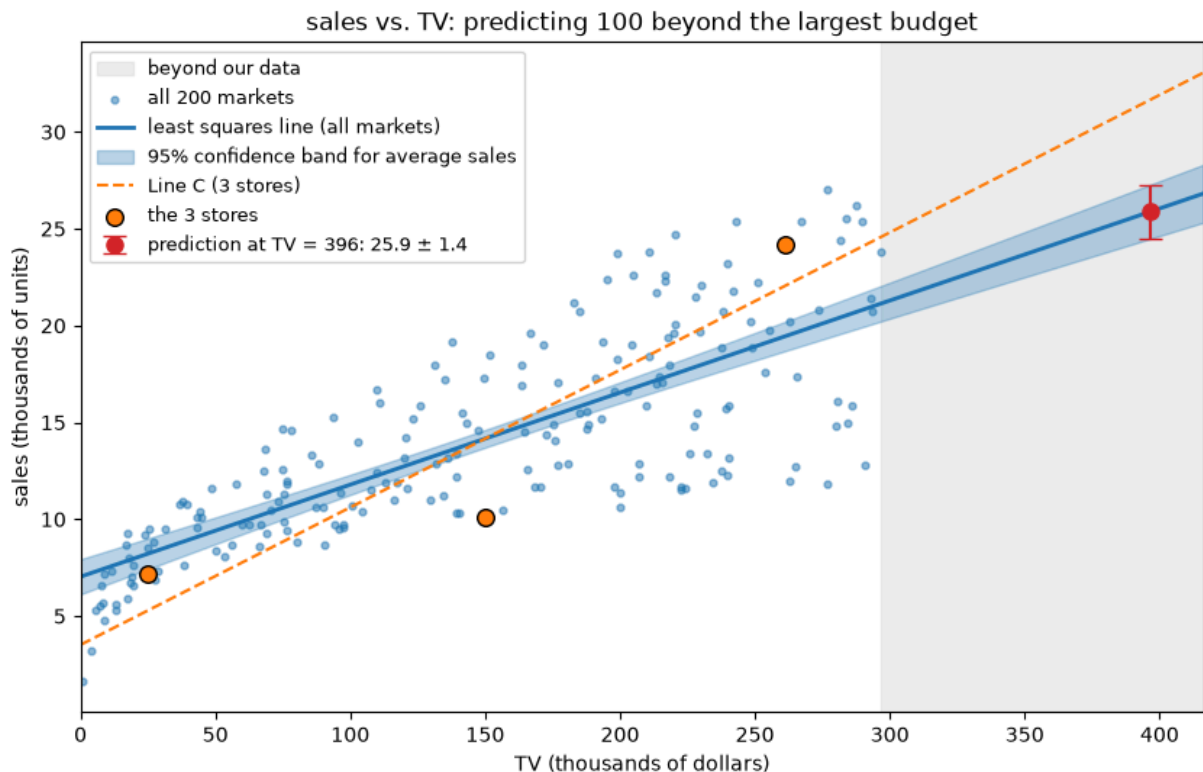
- **3 stores:** about 7 thousand more units, but the 95% interval runs from a large *drop* in sales to a huge gain. It includes 0, which matches the p-value of about 0.25: we can't even rule out "no effect."
- **All 200 markets:** about 4.75 thousand more units, probably between about 4.2 and 5.3 thousand.

▼ Why does the 3-store interval use about 12.7 SEs instead of 2?

"Estimate ± 2 SE" works when there's a decent amount of data. With very few points, the SE is itself a shaky estimate (it's based on only one leftover residual here), so a 95% interval has to be much wider to be safe. The **SEs for 95%** column shows the right multiplier: about 12.7 for 3 points, and about 1.97 (≈ 2) for 200.

Now let's see the prediction itself: sales at the largest TV budget in our data plus \$100k.

```
In [39]: from regression_demos_helpers import plot_tv_decision
plot_tv_decision(Advertising, pts, extra=extra_tv);
```



The shaded band shows where the *average* sales at each budget probably lies (individual markets scatter much more widely). It's narrowest in the middle of our data and widens as we move away, and our prediction sits out in the "beyond our data" region.

How confident, honestly?

The standard error only measures sampling luck, *assuming the straight-line model is right*. It doesn't cover:

- **Extrapolation.** Largest budget + \$100k is beyond every budget in our data. We have no evidence the line keeps going straight out there; returns might level off.
- **Association, not causation.** This is observational data. Markets that spend a lot on TV might differ in other ways (bigger cities, more stores) that also raise sales.
- **Other media.** Radio and newspaper spending also affect sales, and our model ignores them. That's where multiple regression comes in, next time.
- **The spread isn't constant.** The residuals fan out as TV grows, so the standard textbook SEs are a bit optimistic.

Your turn: answer the boss

In 2–3 sentences, answer both of the boss's questions: should we spend \$100k more on TV, and how confident are you?

▼ One possible answer

"Within the budgets we've seen, each extra \$1,000 of TV advertising goes with about 42 to 53 more units sold. An extra \$100k would suggest roughly 4,200 to 5,300 more units, but that's beyond any budget in our data, so treat it as a rough guess. Also, our data show an association, not proof that TV advertising causes the extra sales."

Practice Questions

These questions cover everything *before* the p-value section. Questions like these will appear on the quiz, so try each one before opening its answer. A calculator is fine.

For reference:

- Our three stores, as (TV, sales): (25.0, 7.2), (149.8, 10.1), (261.3, 24.2)
- Line C, the least squares line through them: $\hat{y} = 0.0710x + 3.5155$
- TV is in thousands of dollars; sales is in thousands of units.

Quantitative and graph interpretation

1. Residuals. Using Line C, compute the residual for the store with TV = 149.8. Is that store above or below the line? Does Line C over-predict or under-predict its sales?

▼ Answer

$\hat{y} = 0.0710(149.8) + 3.5155 \approx 14.15$, so $e = y - \hat{y} = 10.1 - 14.15 \approx -4.05$.

The residual is negative, so the store is **below** the line: Line C **over-predicts** its sales by about 4 thousand units.

2. Scoring lines. Line A's residuals are 0, -6.09, and -0.01. Line C's are 1.91, -4.05, and 2.14.

(a) Compute $\sum |e|$ and RSS for each line. (b) Which line wins under each measure? (c) In one sentence, explain why the two measures disagree.

▼ Answer

(a) Line A: $\sum |e| = 6.10$, $\text{RSS} \approx 37.1$. Line C: $\sum |e| = 8.10$, $\text{RSS} \approx 24.6$. (The score table in the notebook shows 37.03 for Line A because it uses unrounded residuals.)

(b) $\sum |e|$ prefers **Line A**; RSS prefers **Line C**.

(c) Squaring punishes big misses much more: Line A's single miss of about 6 costs about 37 in RSS, more than Line C's three moderate misses combined.

3. Loss curves. In the "Sweeping the intercept" figure, the slope is fixed at Line C's value.

(a) Roughly which intercept gives the smallest RSS? The smallest $\sum |e|$? (b) As you drag the slider away from the RSS minimum, what happens to the line in the left panel, and to the RSS?

▼ Answer

(a) RSS is smallest at $b \approx 3.5$ (Line C's intercept, with $\text{RSS} \approx 24.6$). $\sum |e|$ is smallest near $b \approx 5.5$. Even with the same slope, the two losses pick different lines.

(b) The line shifts straight up or down without changing its tilt, and RSS grows, faster and faster the farther you go (the curve is U-shaped).

4. Contour plot. Use the RSS contour plot in "Finding the global minimum."

(a) Where is the best (m, b) ? (b) Which of Lines A, B, and C has the lowest RSS, and how can you tell from the map alone? (c) The contours form a long, tilted valley. What happens to the line as you move along the bottom of that valley?

▼ Answer

(a) At the center of the innermost contour: $m \approx 0.071$, $b \approx 3.5$, which is Line C.

(b) Line C: it sits at the minimum. On a contour map, a point on an inner (lower) contour has a lower RSS than a point on an outer one. Line A is next ($\text{RSS} \approx 37$), and Line B is farther out ($\text{RSS} \approx 101$).

(c) Moving along the valley means raising the slope while lowering the intercept (or the reverse). The line pivots around the middle of the data, so the fit barely changes and RSS stays nearly the same.

5. R^2 and units. For our three stores, $\text{TSS} = 165.4$ and Line C's $\text{RSS} = 24.6$.

(a) Compute R^2 . (b) TSS is the RSS of which simple model? (c) If sales were recorded in units instead of thousands of units, what would happen to TSS – RSS? To R^2 ?

▼ Answer

(a) $R^2 = 1 - \frac{24.6}{165.4} \approx 0.851$.

(b) The "mean only" model: a flat line at $\bar{y} \approx 13.83$ that ignores TV.

(c) Every residual gets multiplied by 1000, so every squared residual gets multiplied by $1000^2 = 10^6$. TSS – RSS goes from about 140.8 to about 140.8 million. R^2 is a ratio, and TSS and RSS scale by the same factor, so it stays 0.851.

6. Slopes, units, and predictions.

(a) Interpret Line C's slope, 0.0710, in words, with units. (b) Use Line C to predict sales at TV = 296.4 (the largest budget in the data) and at TV = 396.4. How much bigger is the second prediction?

▼ Answer

(a) Each extra \$1,000 of TV advertising goes with about 0.071 thousand, or **71, more units sold**.

(b) $0.0710(296.4) + 3.5155 \approx 24.55$ and $0.0710(396.4) + 3.5155 \approx 31.65$ thousand units. The difference is $100 \times 0.0710 = 7.1$ thousand units: slope $\times$ change in TV.

7. Matching plots to R^2 . The five plots in "A Few Cases" have these R^2 values, in scrambled order: 0.00, 0.06, 0.10, 0.21, 0.88. Cover up the plot titles, match each value to its plot (baseline, noisy y , wrong functional form, narrow range of x , outliers in y), and explain why each low one is low.

▼ Answer

- **Baseline: 0.88.** A straight line with a little noise; the line explains most of the variation.
- **Noisy y : 0.10.** The right shape, but lots of scatter that x can't explain.
- **Wrong functional form: 0.00.** The truth is a symmetric curve, so the best straight line is nearly flat and explains almost nothing.
- **Narrow range of x : 0.06.** Over such a short range of x , the trend barely moves y compared with the noise.
- **Outliers in y : 0.21.** Three extreme points add huge squared residuals and pull the line toward them.

Conceptual

8. Why not the plain sum of residuals? Line D has $\sum e = 0$, the same as Line C. Why does that make the plain sum of residuals a bad way to score a line?

▼ Answer

Positive and negative residuals cancel. Line D misses by -12.65 , -3.51 , and $+16.16$, yet those sum to 0. In fact, every line through the mean point $(\bar{x}, \bar{y})$ has

$\sum e = 0$, no matter how badly it fits. We need every miss to count as "bad," whatever its sign.

9. $\sum |e|$ vs. RSS. Both make every miss count. How do they differ in which lines they prefer? Why do we call choosing between them a *choice*?

▼ Answer

RSS squares each miss, so it punishes big misses much more: one miss of 6 costs 36, while three misses of 2 cost only 12. RSS prefers lines that spread their errors around (Line C); $\sum |e|$ is more willing to accept one big miss (Line A).

Neither one is "correct" by some law of nature; they measure badness differently. RSS (least squares) is the standard choice, partly because there's a simple formula for the line that minimizes it.

10. One parameter at a time. The loss explorers varied the intercept with the slope fixed, then the slope with the intercept fixed. Why isn't that enough to find the best line in general?

▼ Answer

The best intercept depends on which slope you've fixed, and vice versa. If the fixed value isn't already the best one, each sweep only finds the best line *given that value*. The truly best line is the lowest point of the whole RSS surface over both parameters at once, which is what the 3-D surface and contour plots show. (Our sweeps found Line C only because we fixed each parameter at Line C's value.)

11. What R^2 measures. What does R^2 tell you? Why do we use the fraction $\frac{\text{TSS}-\text{RSS}}{\text{TSS}}$ instead of just the difference $\text{TSS} - \text{RSS}$?

▼ Answer

R^2 is the fraction of the variance in y that the model removes, compared with always predicting the mean $\bar{y}$ (the "variance explained"). 0 means no better than predicting the mean; 1 means a perfect fit.

The raw difference depends on the units of y : switching **sales** from thousands of units to units multiplies it by a million, even though the fit is exactly as good. The fraction doesn't depend on units.

12. Low R^2 . A classmate says, "My R^2 is 0.10, so there's no relationship between x and y ." Using the "A Few Cases" examples, explain why that conclusion doesn't follow.

▼ Answer

In the noisy example, there *is* a real straight-line relationship, and the fitted line is close to the true line, yet R^2 is only 0.10: the noise is just large compared with the trend. A low R^2 can also come from a strong relationship with the wrong shape (the curved example has $R^2 \approx 0$), too narrow a range of x , or a few outliers. R^2 measures how much of the variation the line explains, not whether a relationship exists.

13. Extrapolation. Using Line C, why is a prediction at TV = 396.4 riskier than a prediction at TV = 200, even though both come from the same line?

▼ Answer

TV = 200 is within the range of budgets in our data. TV = 396.4 is beyond every budget we've observed (the largest is 296.4), so we have no evidence the straight line keeps holding out there; sales might level off, for example. The formula happily produces a number either way, but the data only support it within the range they cover.